深圳河套學院
Shenzhen Loop Area Institute

Natural Language Processing and Large Language Models

Xiaojun Quan and Zhengjun Yue

Shenzhen Loop Area Institute

Autumn, 2026

Lecture 5

Pretrained Language Models

Learning Objectives

By the end of this lecture, you should be able to:

1. Explain the role of pretraining in learning general-purpose language representations.
2. Understand BERT, including bidirectional encoding and masked language modeling.
3. Describe encoder–decoder models and how they support seq-to-seq tasks.
4. Explain GPT models, including causal language modeling and autoregressive generation.
5. Compare the three model architectures in terms of information flow, training objectives, and applications.

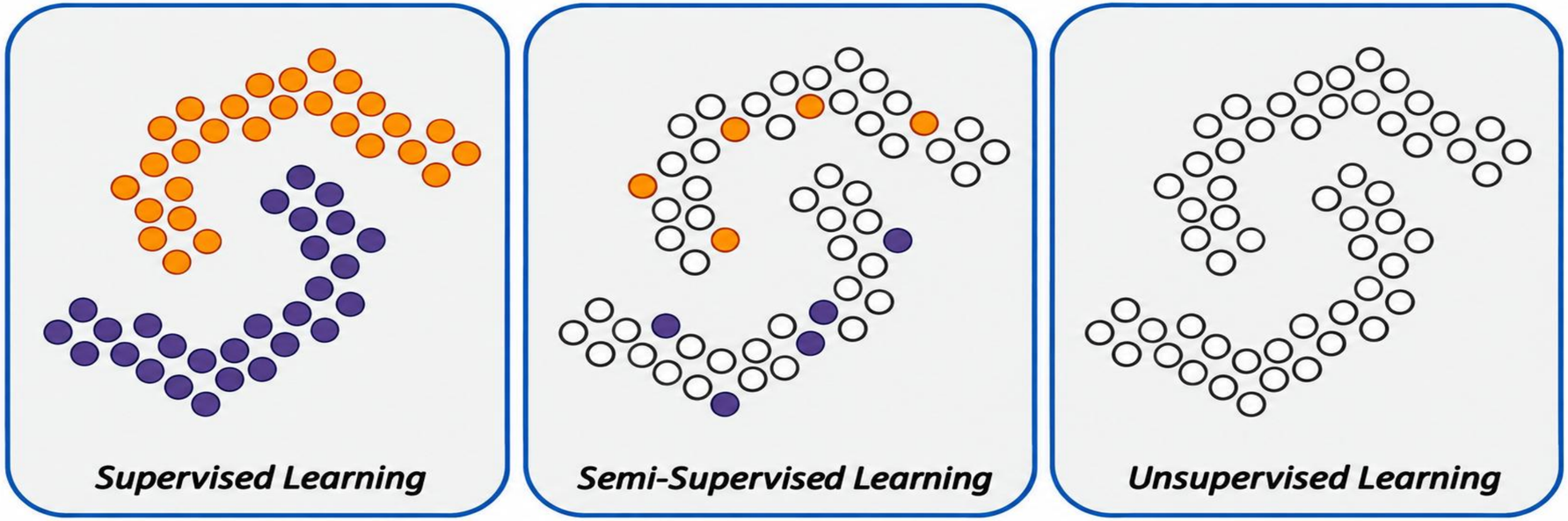
Outline

- **Pretraining**
- Groundbreaking: BERT
- Encoder-Decoder Models
- Decoder-Only: GPTs
- Practice & Discussion

Traditional machine learning paradigm

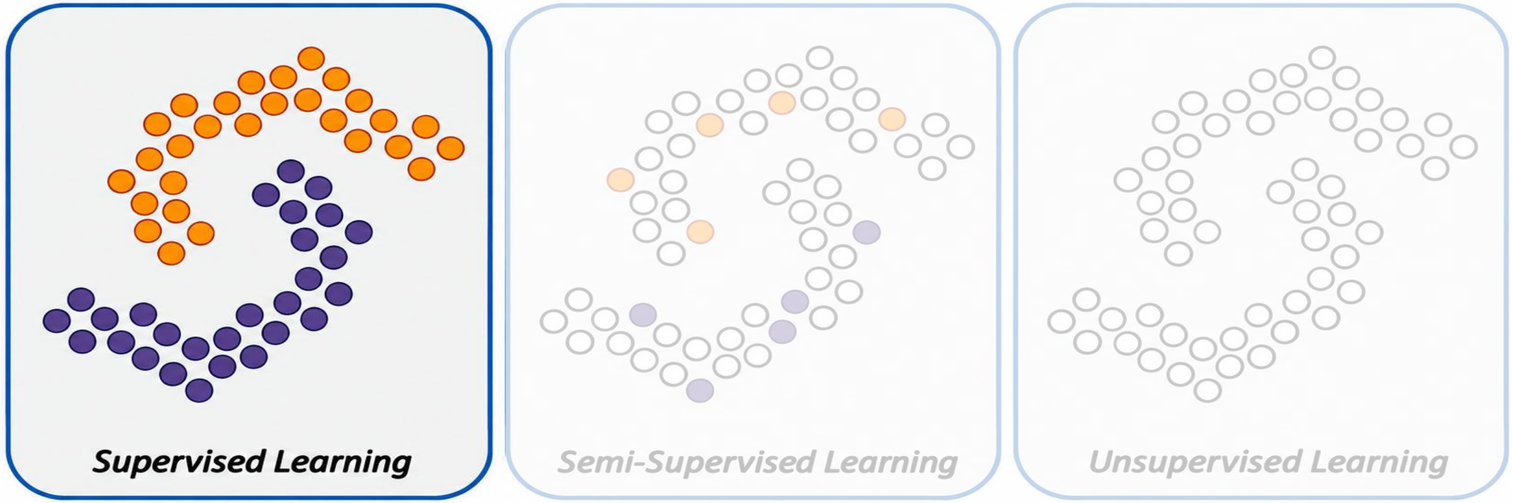
Machine learning paradigm

- We usually divide the machine learning methods into three basic paradigms according to whether the training data has *labels*: supervised learning, unsupervised learning, and semi-supervised learning.



Traditional machine learning paradigm

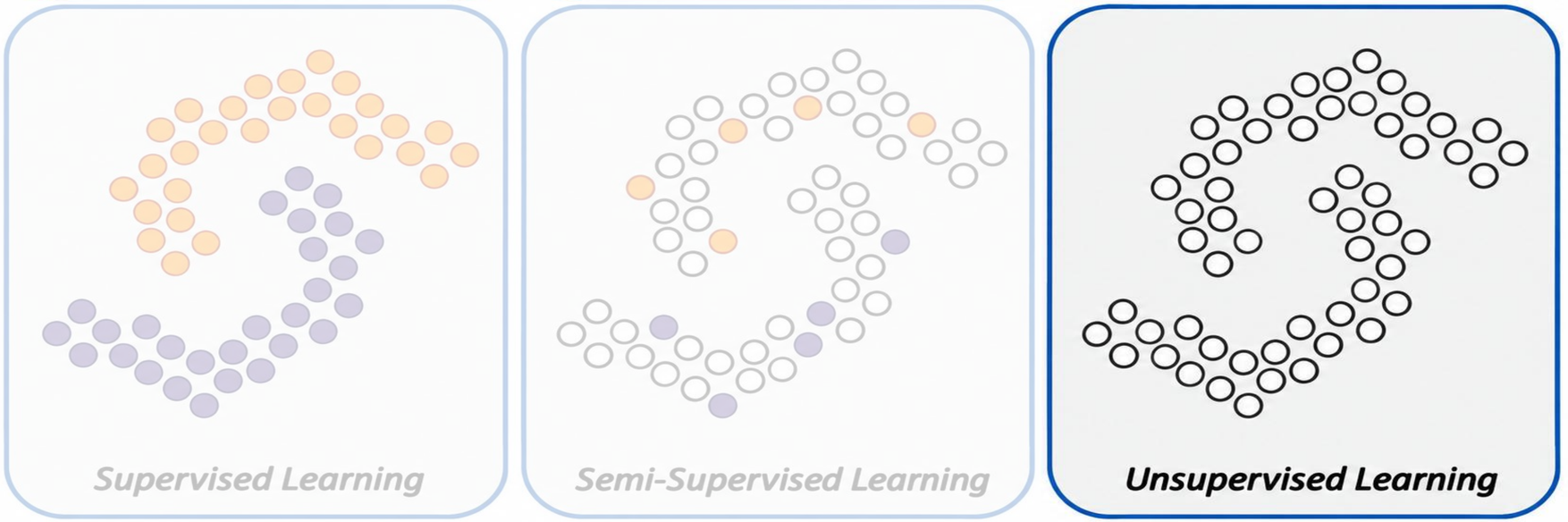
- What is Supervised Learning?
  - The core idea is to use *labeled* data to train models, enabling models to make predictions on unknown data.



- Common algorithms: *logistic regression*, *support vector machine* (SVM), *neural networks*.

Traditional machine learning paradigm

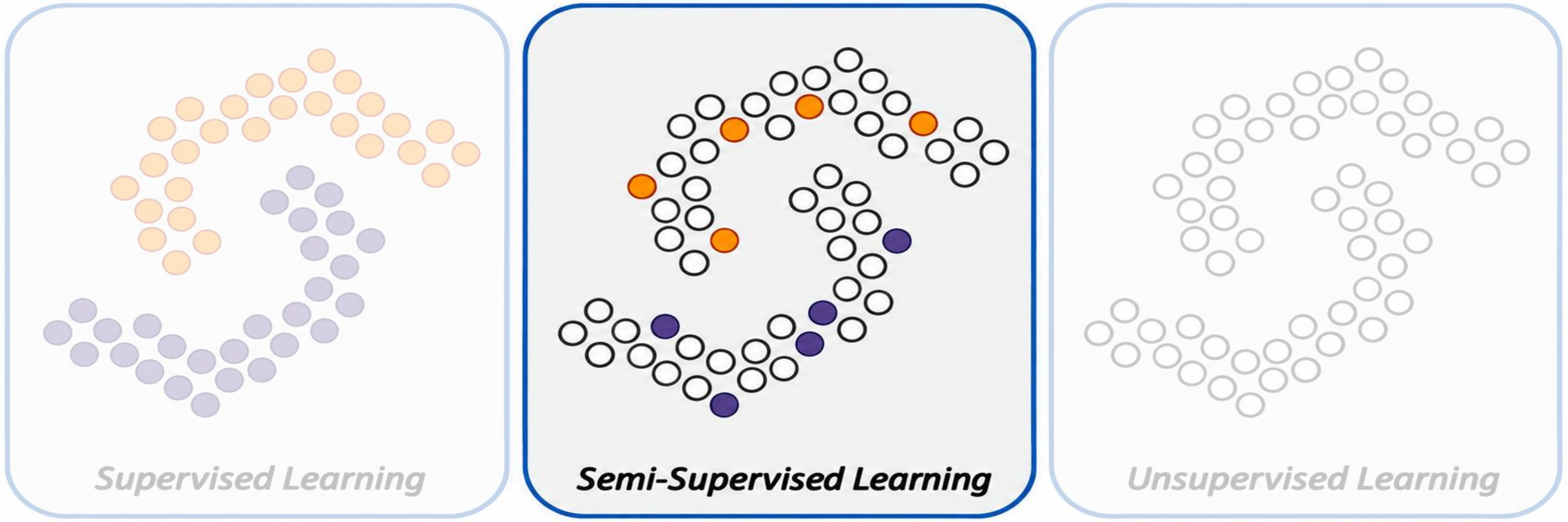
- What is Unsupervised Learning?
  - *Does not rely on labels*. Its goal is to discover *structures*, *patterns*, or *relationships* from unlabeled data.



- Common algorithms: *clustering*, *dimensionality reduction*.

Traditional machine learning paradigm

- Semi-Supervised Learning?
  - A method between supervised and unsupervised learning. It combines a small amount of *labeled* data and a large amount of *unlabeled* data to train models.

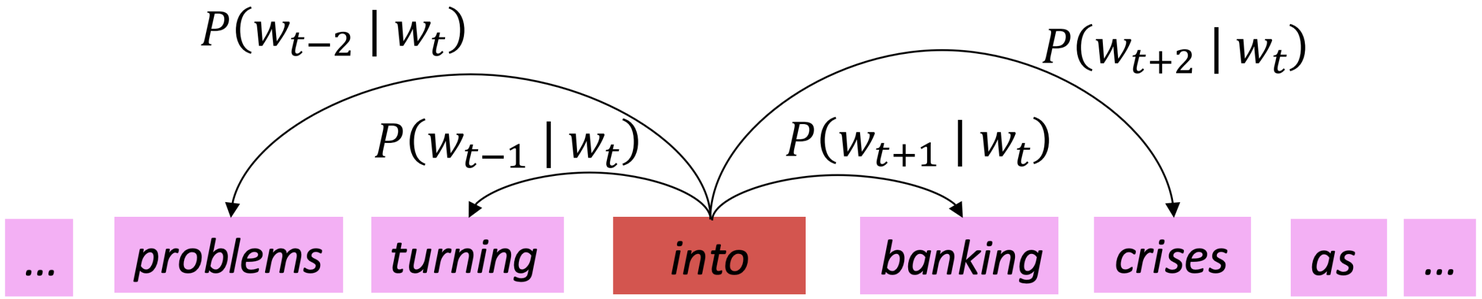


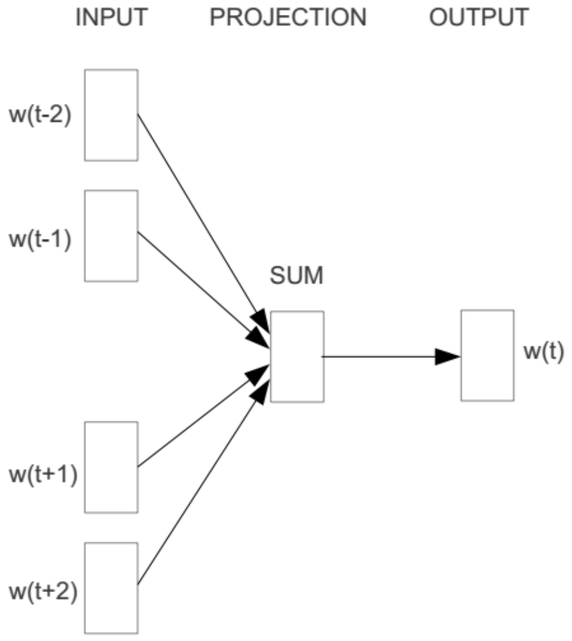
- Common algorithms: *pseudo-labeling*, *self-training*.

Self-Supervised Learning

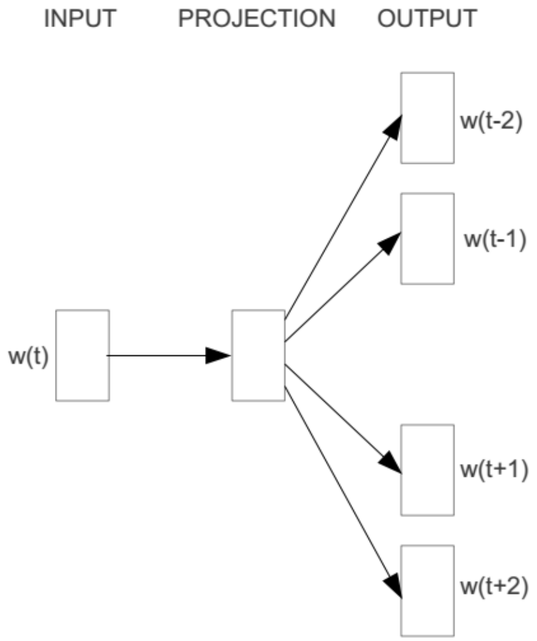
- Self-Supervised Learning does not rely on labeled data, but automatically generates “supervision signals” (pseudo-labels) from the data itself for training.
- **Idea**: Let the model predict a part of the data from the data itself to learn the structure or semantics of the data.
  - Input: raw unlabeled data samples.
  - Goal: construct “proxy tasks” using the data itself.
  - Output: general features or representations that can be transferred.

Self-Supervised Learning: Word2Vec





CBOW



Skip-gram

Self-Supervised Learning: More Examples

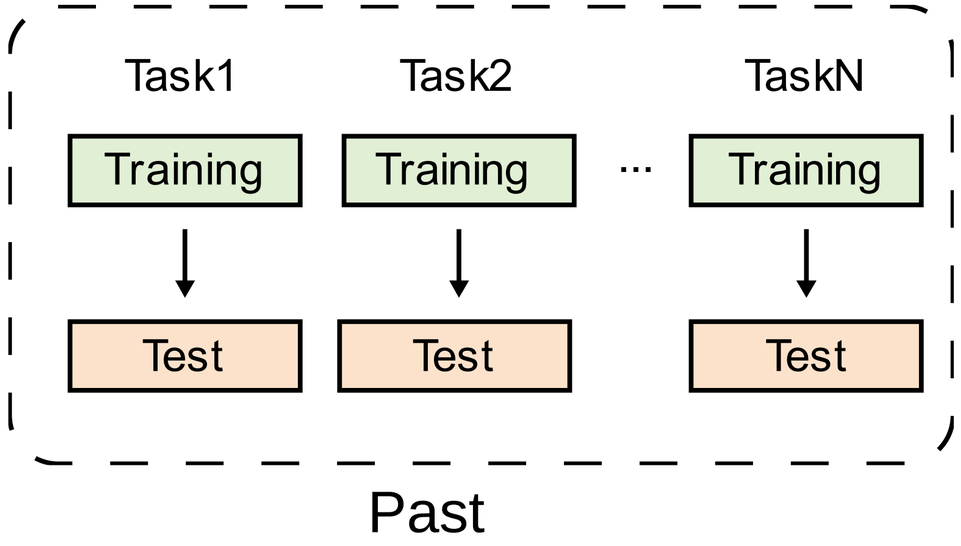
- **NLP example**
  - Input sentence: "I went to the [MASK] today."
  - Task: Predict the masked word (e.g., "library")
  - Model learns to understand language structure, grammar, and context.
- **Computer Vision example**
  - Raw image → different data augmentations (rotation, cropping, etc.)
  - Task: recognize that these augmented versions are "**the same image**".
  - The model learns to extract semantic similarity and structural features.

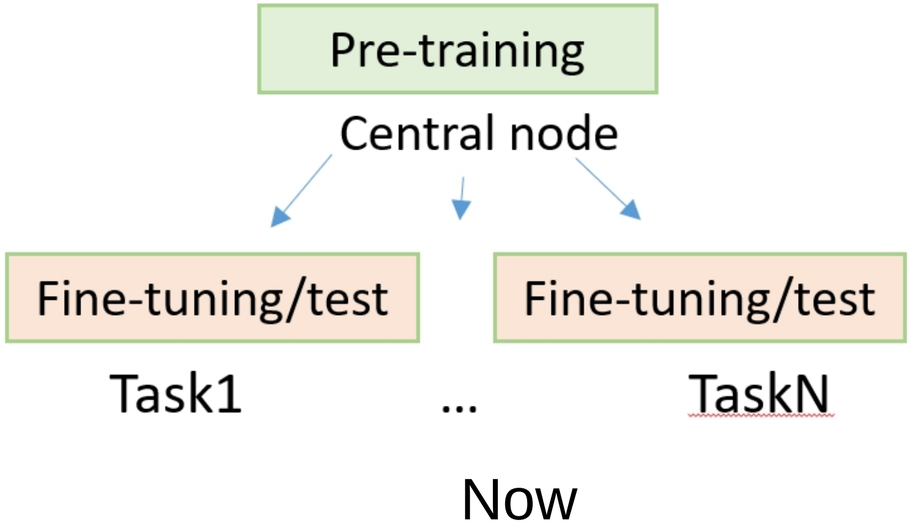
Self-Supervised Learning: Advantages

- **Reduce reliance on *manually labeled data***
  - Offers the ability to generate an unlimited amount of training data.
- **Enhance *feature representation* capabilities**
  - Improve performance in downstream tasks.
- **Support the development of large-scale models**
  - Foundation models such as *ChatGPT* and *DeepSeek* are based on self-supervised pretraining.
  - Critical for the *scaling law* of LLMs.

Pretraining

- What is Pretraining?
  - Training a model in advance on a large-scale dataset, and then using its parameters for the *initialization* phase of other specific tasks.



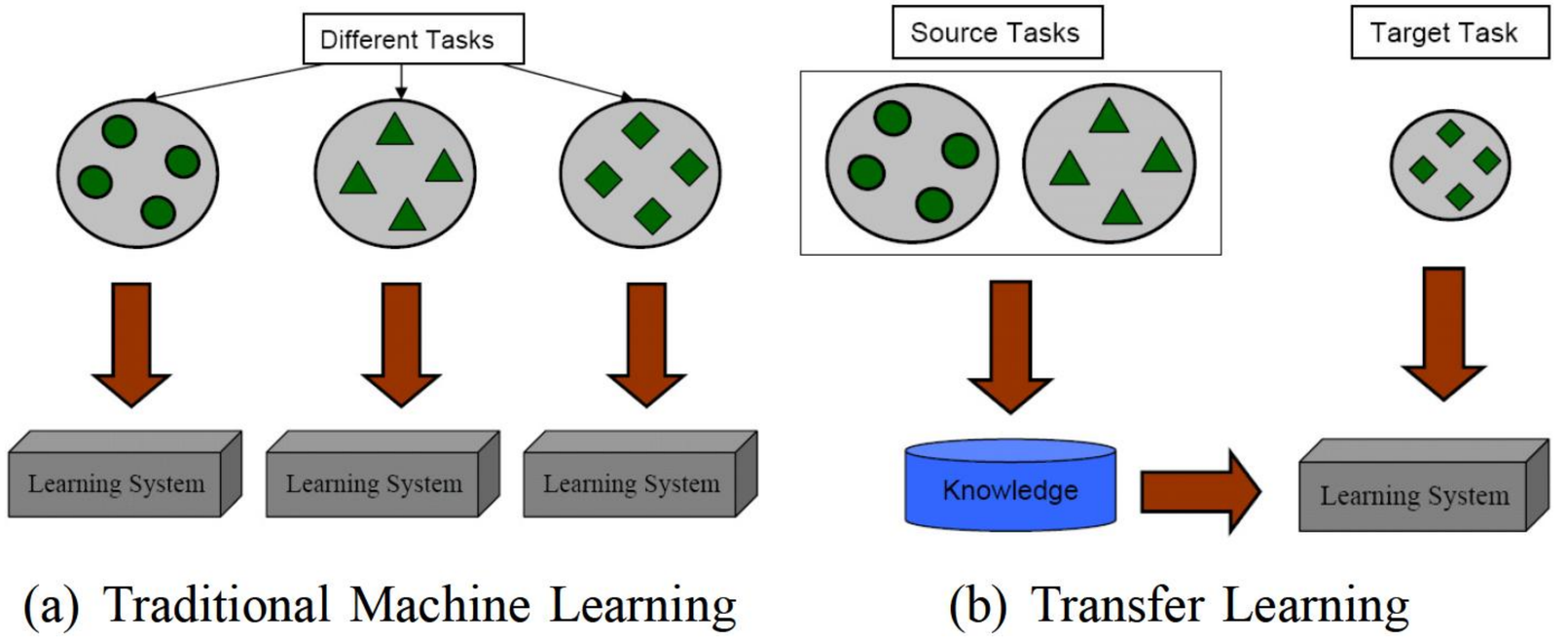


Why Pretraining?

- Pretraining on large-scale corpora enables the learning of general language representations, which facilitates the completion of downstream tasks.
- Pretraining provides better model initialization, which usually leads to stronger generalization performance and accelerates convergence (加速收敛) on target tasks.
- Pretraining can also be regarded as a regularization (正则化) method to avoid overfitting on small datasets.

Related Concept: Transfer Learning

- **Transfer Learning** is a machine learning technique that refers to transferring and reusing knowledge (or models) learned from one task to another related task to improve the performance of that task.



(a) Traditional Machine Learning (b) Transfer Learning

Related Concept: Transfer Learning

- **Pretrained Word Embeddings:** Word embeddings pretrained on large-scale corpora can capture semantic and syntactic (语义和语法) information of words, and can be directly used for various NLP tasks.

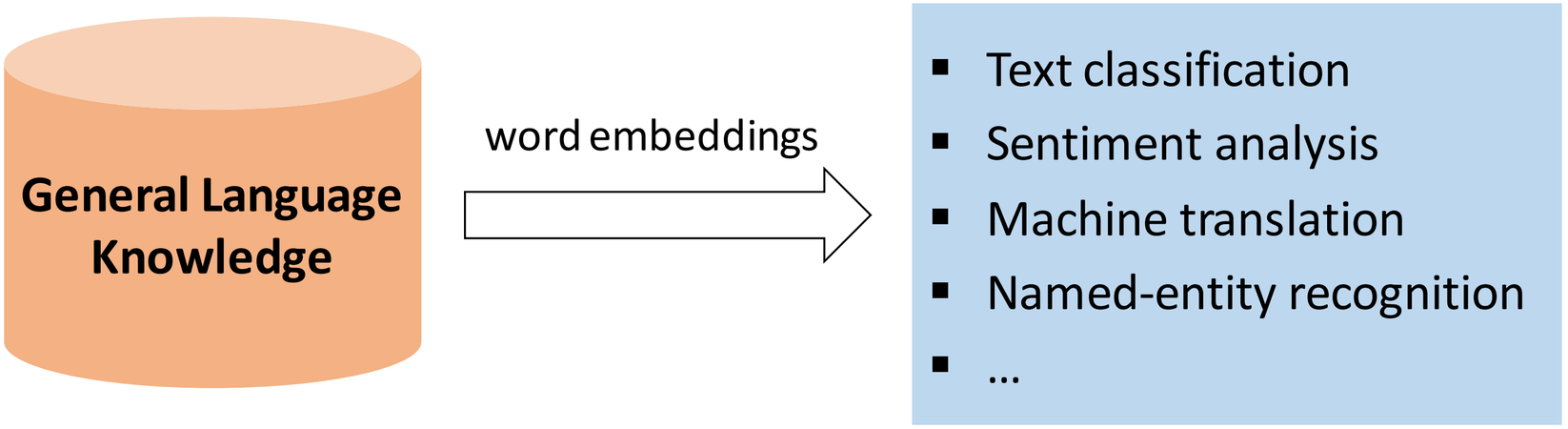
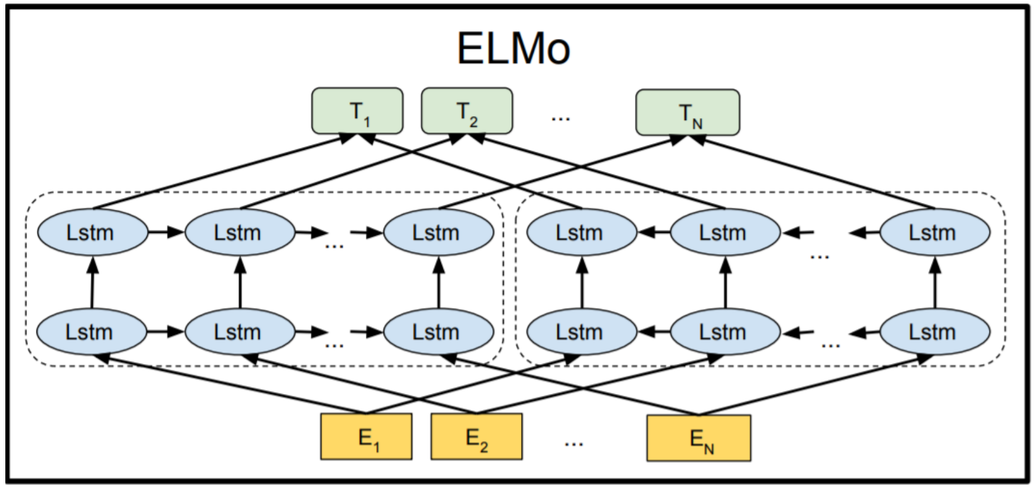
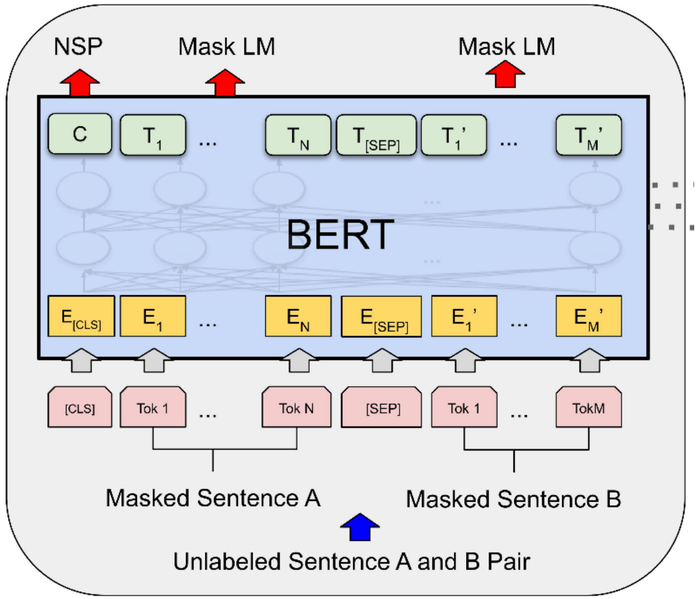


Figure: Transfer from general language knowledge to specific NLP tasks

Related Concept: Transfer Learning

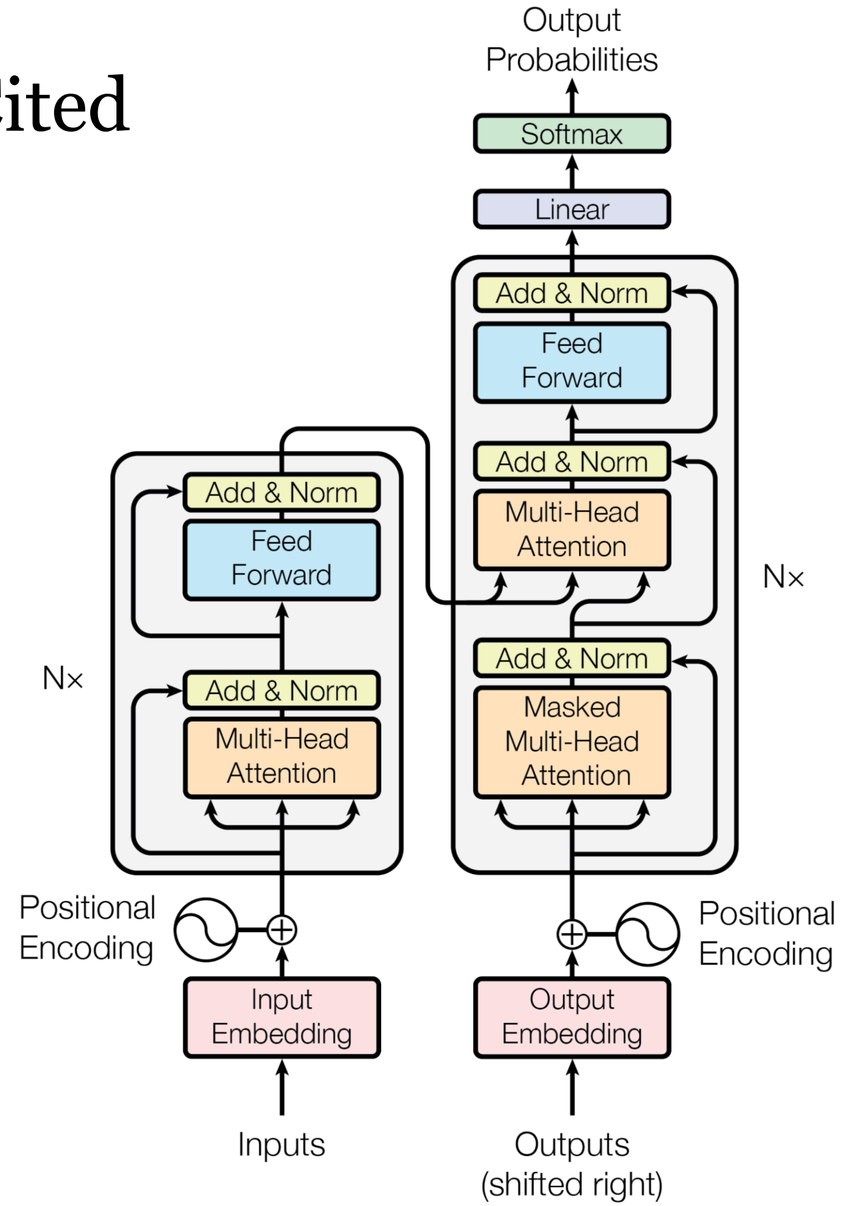
- **Word Embeddings:** A global and static representation, which cannot reflect its semantic changes in different contexts.
- **Contextual Word Embeddings:** Dynamically generate dense vector representations according to the context for each word.
  - Examples: ELMo, BERT





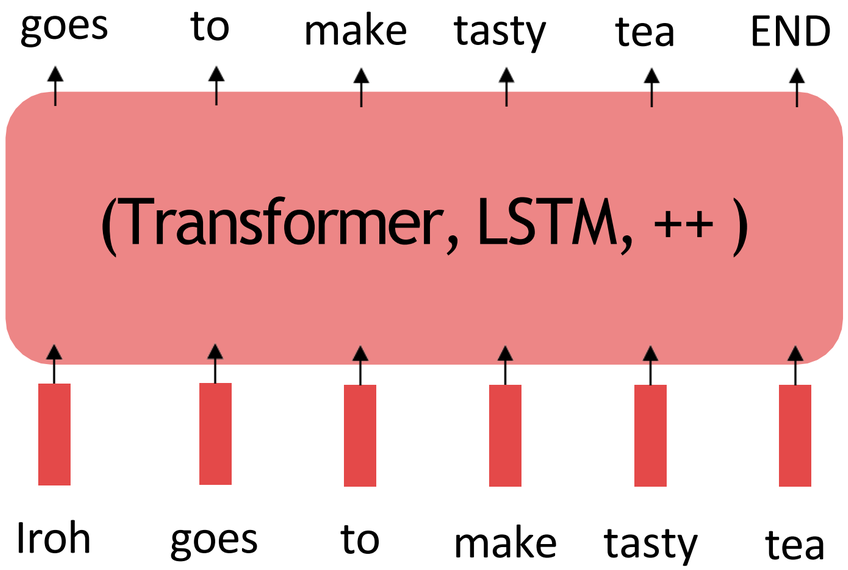
Related Concept: Transformer

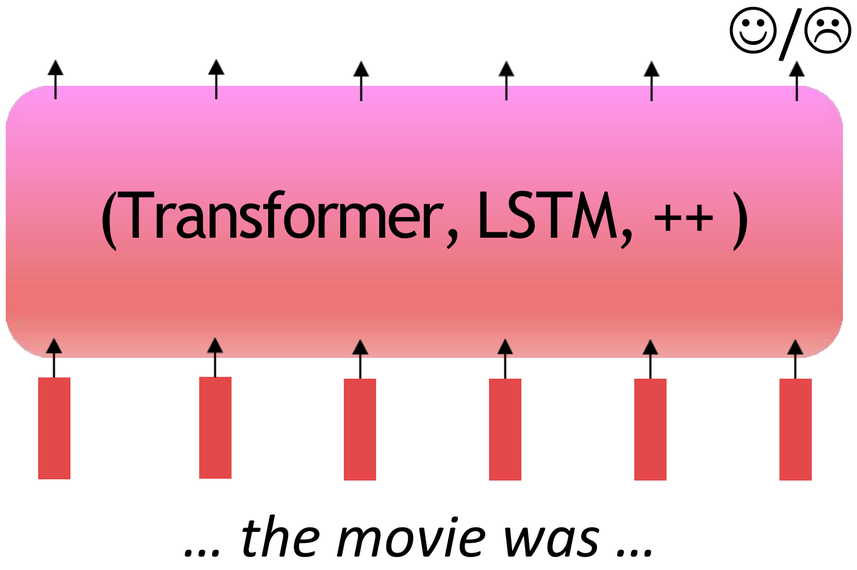
- **Transformer**: Attention is all you need (Cited by 275,368, 2026/10/12).
  - Utilize self-attention
  - Simple attention scoring function
  - Added linear layers for Q, K and V
  - Multi-head attention
  - Added positional encoding
  - Added residual connection
  - Simulate decoding by masking



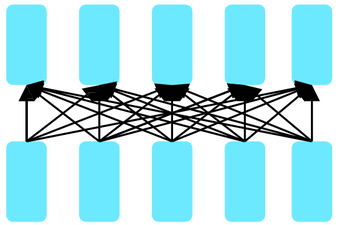
Related Concept: Fine-tuning

- **Step 1**: Pretraining
  - Use a large amount of text to learn general knowledge!
- **Step 2**: Fine-Tuning
  - Use little labeled data; the model adapt to specific tasks!



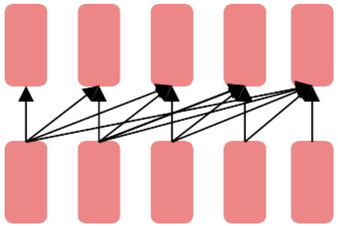


Three Structures for Pretrained Models



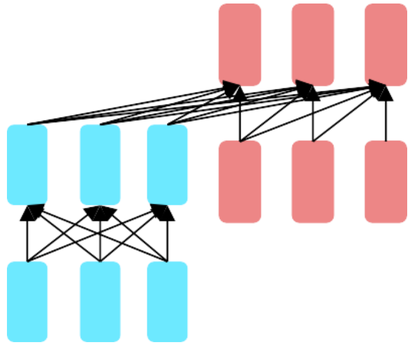
Autoencoder

- Use the *encoder* of Transformer.
- Obtain *bidirectional* context.
- Example: BERT and its various variants.



Autoregressive

- Use the *decoder* of Transformer.
- Obtain *undirectional* context.
- Example: GPT and its various variants.

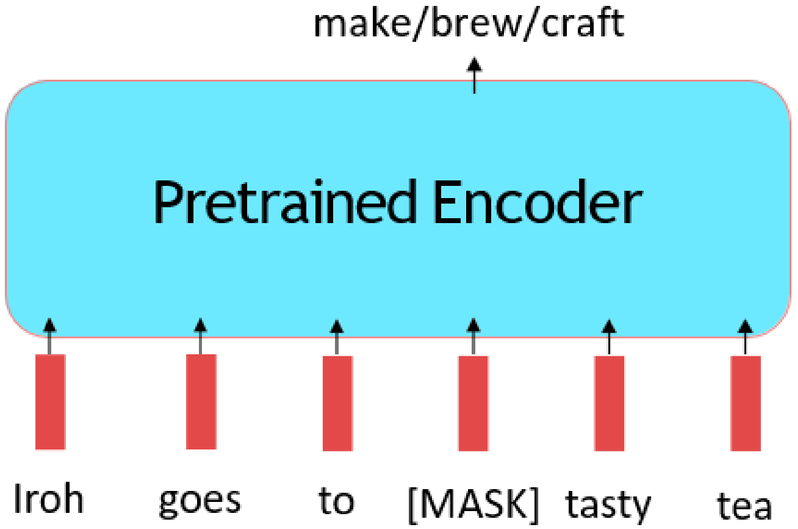


Encoder-Decoder

- Use the *encoder* and *decoder* of Transformer.
- Use pairs of samples for pretraining.
- Example: T5, BART, standard Transformer.

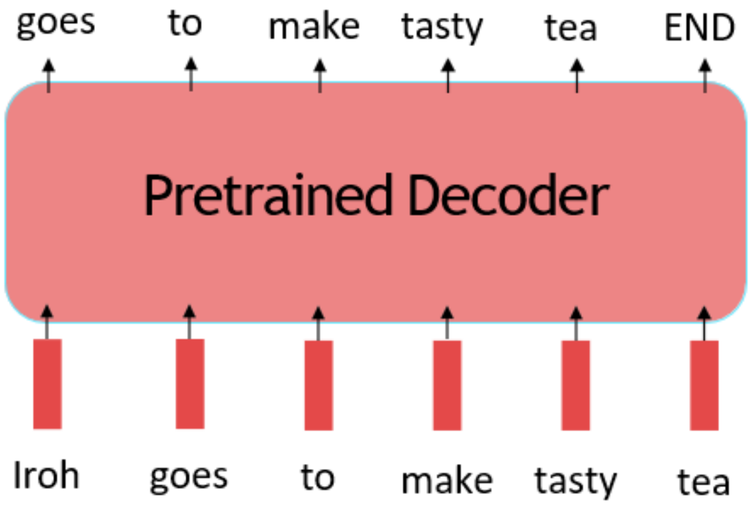
Pretrained Models: Autoencoder

- Autoencoder Model: Uses only the *encoder* part of Transformer.
  - *Autoencoder-based pretraining* reconstructs original tokens from ***corrupted*** inputs, i.e., "fill-in-the-blank" tasks.
  - *Autoencoder models* are usually used for content **understanding** tasks, such as: *text classification*, *extractive question answering*, etc.
  - BERT



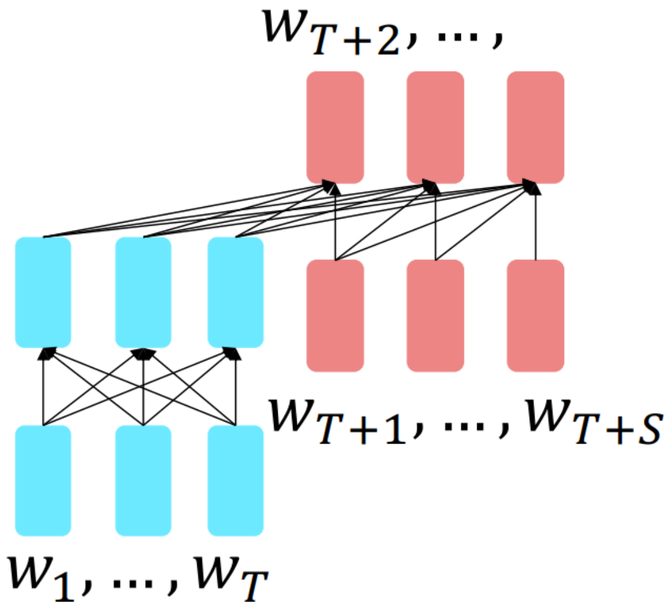
Pretrained Models: Autoregressive

- Autoregressive Model: Uses only the *decoder* part of Transformer.
  - *Autoregressive-based pretraining* uses an ***autoregressive*** approach to predict the next token based on the existing token sequence.
  - *Autoregressive models (AR models)* are usually used for **generation** tasks, such as: *machine translation*, *generative question answering*.
  - GPT series models



Pretrained Models: Encoder-Decoder

- Encoder-Decoder Model: Uses both the encoder and decoder parts of the Transformer.
  - *Encoder-decoder based pretraining* treats each task as a sequence-to-sequence generation problem.
  - *For example*, for text classification tasks, the encoder reads text as input, and the decoder generates label text instead of directly classifying.
  - *Encoder-decoder models/seq2seq models* are usually used for tasks that require both ***understanding*** and ***generation***, such as: machine translation.

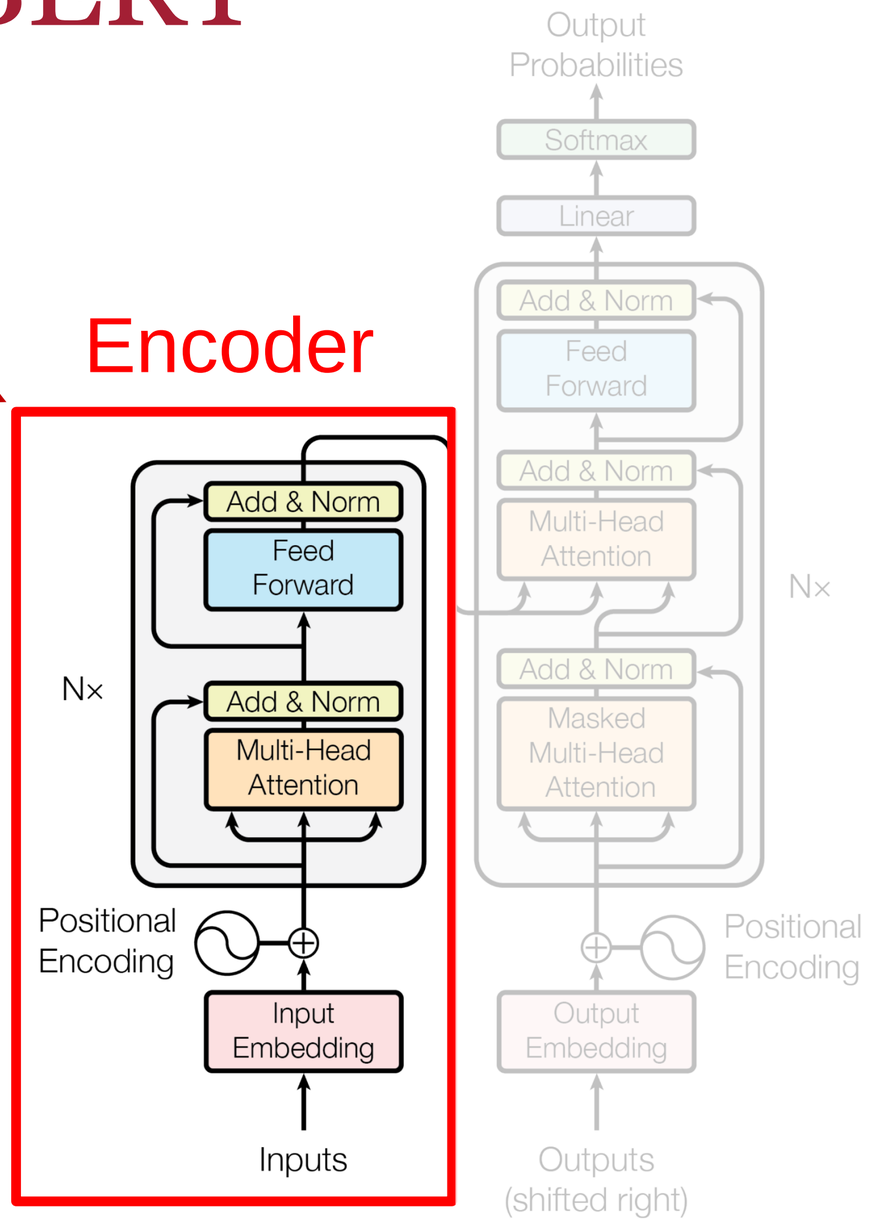


Outline

- ❑ Pretraining
- ❑ **Groundbreaking: BERT**
- ❑ Encoder-Decoder Models
- ❑ Decoder-Only: GPTs
- ❑ Practice & Discussion

Pretrained Encoder Models: BERT

- **BERT**: Bidirectional Encoder Representations from Transformers
  - Definition: Transformer encoder-based pretrained model, learns *bidirectional* semantic information from context.
  - Training: Uses large-scale unsupervised corpus for pretraining (captures context), then adapts to *downstream NLP tasks*.



Pretrained Encoder Models: BERT

- **Core**: Deep Transformer Encoder + bidirectional context learning.

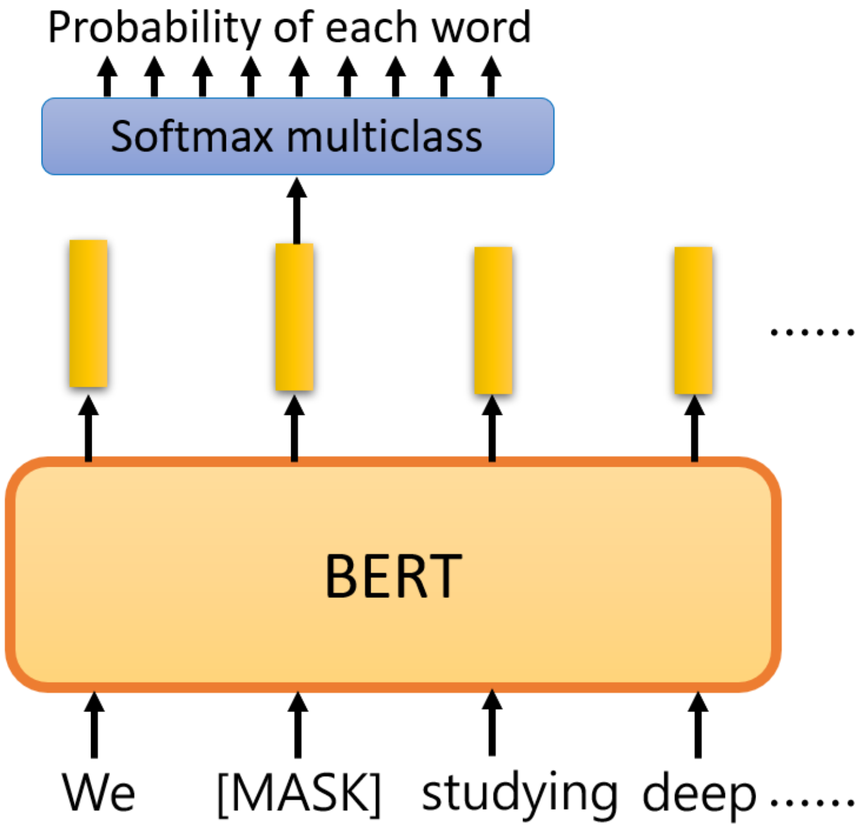
Question: Why bidirectional?

Answer: Understanding word meaning requires both left and right context.

- **Example**:
  - "We went to the river bank." (bank = riverside)
  - "I need to go to bank to make a deposit." (bank = financial institution)

Pretrained Encoder Models: BERT

- **Two pretraining tasks**
  - Task 1: Masked Language Modeling (MLM)
    - Randomly mask 15% of tokens in the input; model predicts masked tokens.
  - Why MLM?
    - Traditional left-to-right models only use *unidirectional* context; MLM enables bidirectional encoding.



BERT: MLM Details

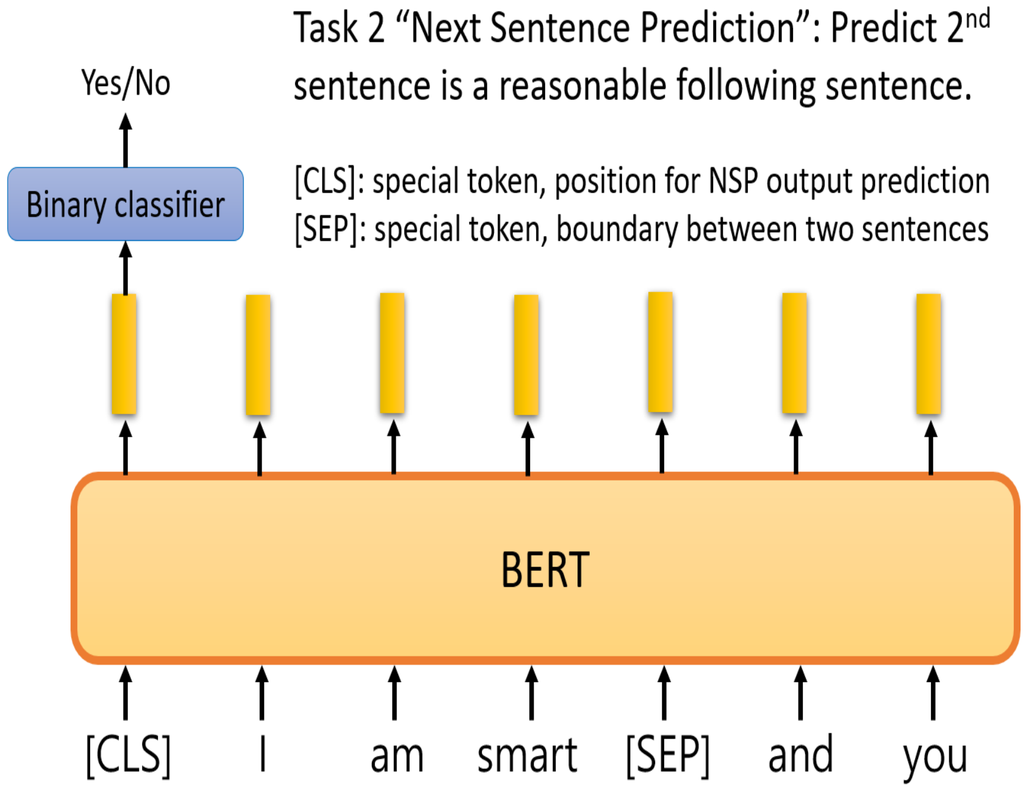
- **15% token prediction rules**
  - 80%: Replace with [MASK] (e.g., “go to store" → “go to [MASK]").
  - 10%: Replace with random word (e.g., “go to store" → “go to running").
  - 10%: Keep original word (still needs prediction, e.g., “go to store" → “go to store").
- Why?
  - Prevent over-reliance on [MASK], as it doesn’t appear in fine-tuning.

BERT: MLM Details

Description	Percentage	What the model *sees*	What the label is
Select tokens for prediction	**15% of all tokens**	Token may be masked /changed/unchanged	Original token
Replace selected token with **MASK**	**80% of the 15%**	MASK	Original token
Replace selected token with **a random word**	**10% of the 15%**	Random token	Original token
Leave selected token **unchanged**	**10% of the 15%**	Original token	Original token
All non-selected tokens	**85%**	Unchanged	No prediction

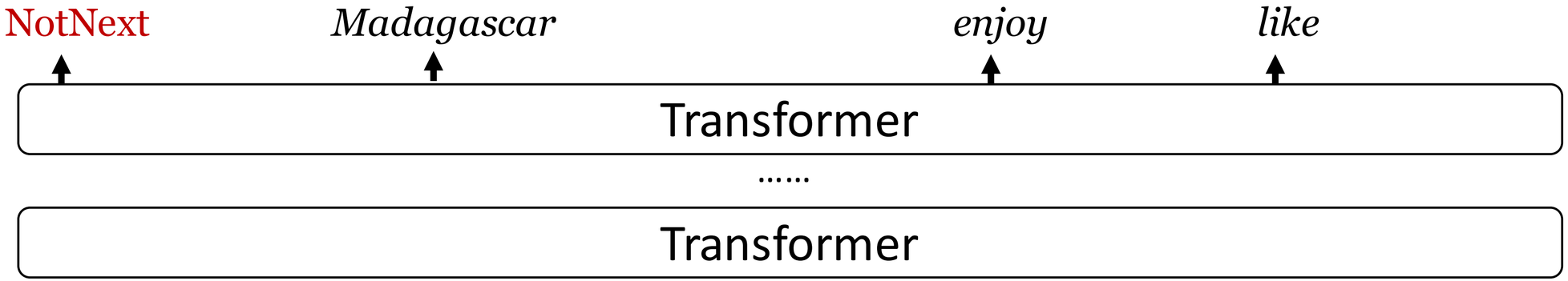
Pretrained Encoder Models: BERT

- Two Pretraining Task
  - Task 2: Next Sentence Prediction (NSP)
    - Predict if *the second sentence is the real successor (继任者) of the first* in the corpus.
  - Motivation
    - Many downstream tasks (natural language inference, paraphrase detection, QA) require understanding sentence relations;
    - Narrows gap between *pretrain* and *fine-tune*.



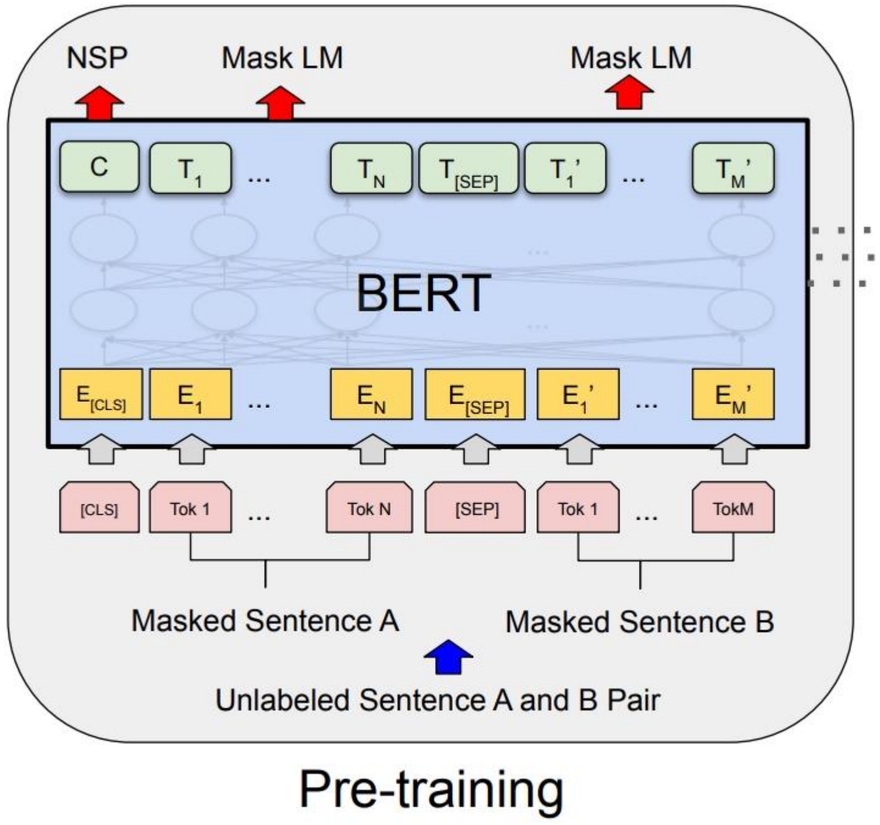
BERT: NSP Details

- Data Construction:
  - 50%: second sentence = real successor;
  - 50%: second sentence = random from corpus.
- Input format:
  - [CLS] Text chunk 1 [SEP] Text chunk 2

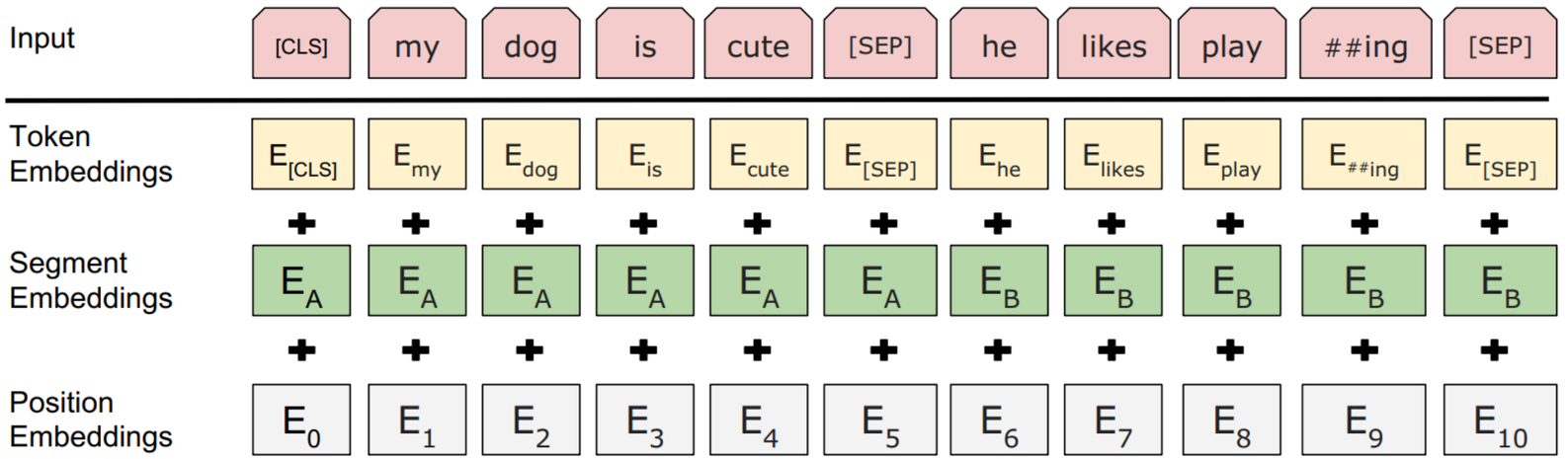


BERT: Pretraining

- Pretraining: MLM + NSP



Pre-training

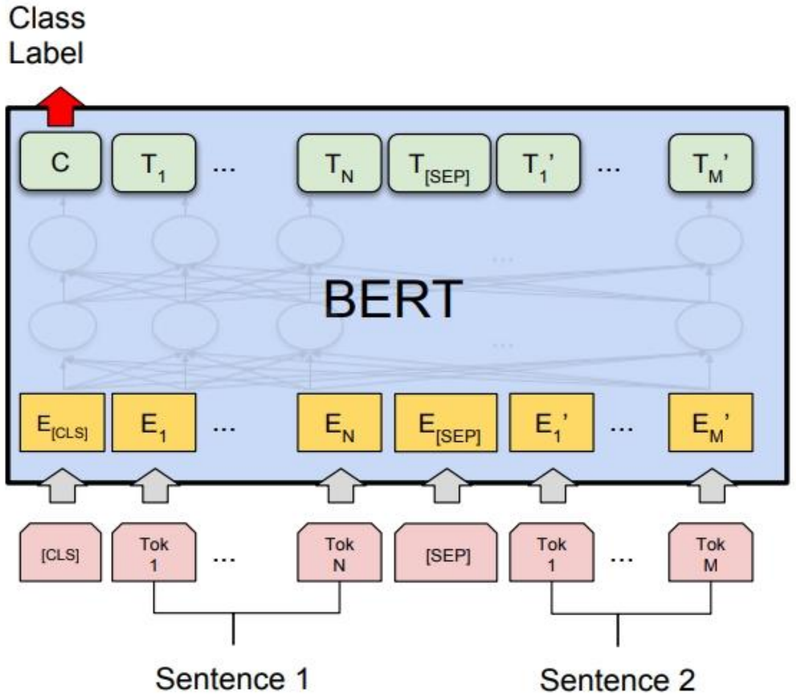


Segment Embedding: Two learnable vectors (for Sentence A/B) to mark token's sentence affiliation (隸屬).

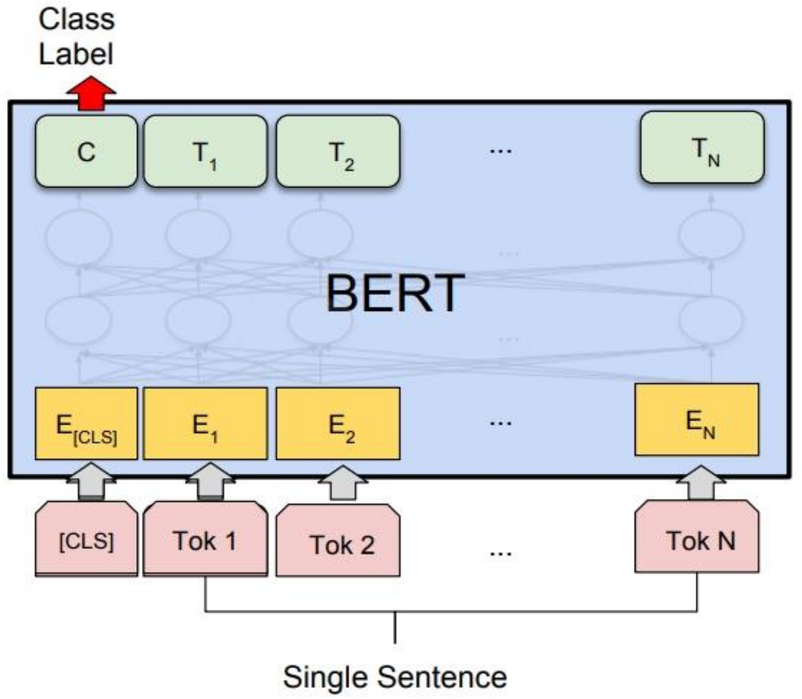
BERT Details

- **Model Sizes**
  - BERT-base: 12 layers, 768-d hidden state, 12 attention heads, 110M params.
  - BERT-large: 24 layers, 1024-d hidden state, 16 attention heads, 340M params.
- **Training Data**
  - BooksCorpus (800M words).
  - English Wikipedia (2.5B words).
- **Training Resources**: 64 TPU chips, 4 days total.
- **Fine-tuning**: Runs on single GPU (one pretraining, multiple fine-tunings).

Applications: Sentence-Level Tasks



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA

- Segment embeddings added.
- Classifier has $C \times h$ new params (C=num classes, h=hidden dim).

Applications: Sentence-Level Tasks

- Sentence Pair Classification (e.g., MNLI, QQP)

MNLI

Premise: A soccer game with multiple males playing.

Hypothesis: Some men are playing a sport.

{entailment, contradiction, neutral}

QQP

Q1: Where can I learn to invest in stocks?

Q2: How can I learn more about stocks?

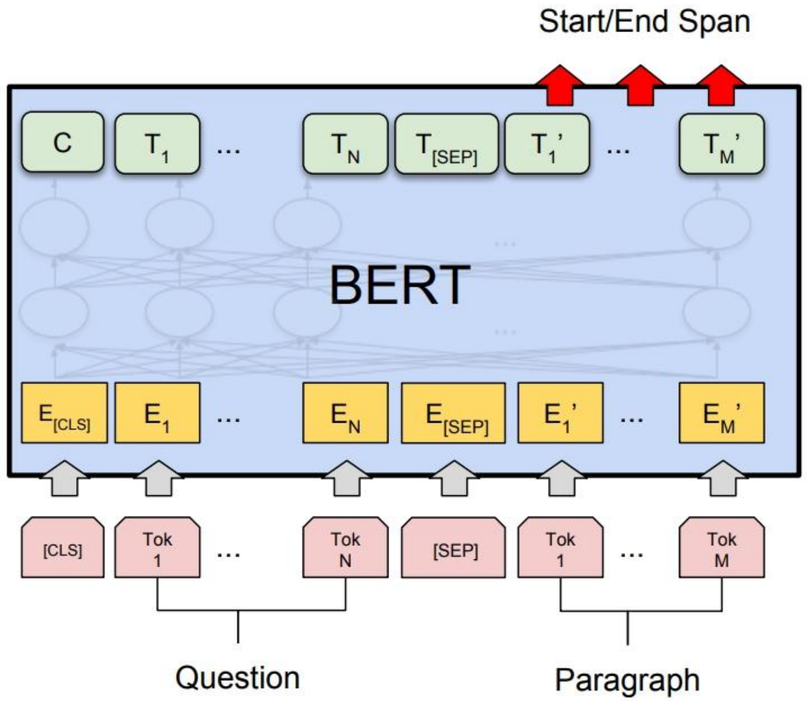
{duplicate, not duplicate}

- Single Sentence Classification (e.g., SST-2, CoLA)

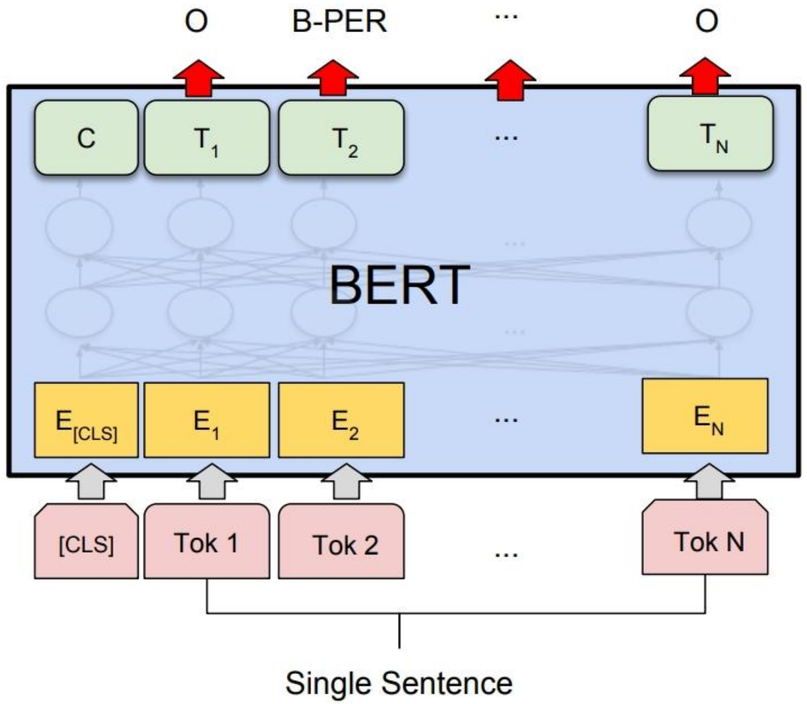
SST2 rich veins of funny stuff in this movie

{positive, negative}

Applications: Token-Level Tasks



(c) Question Answering Tasks: SQuAD v1.1

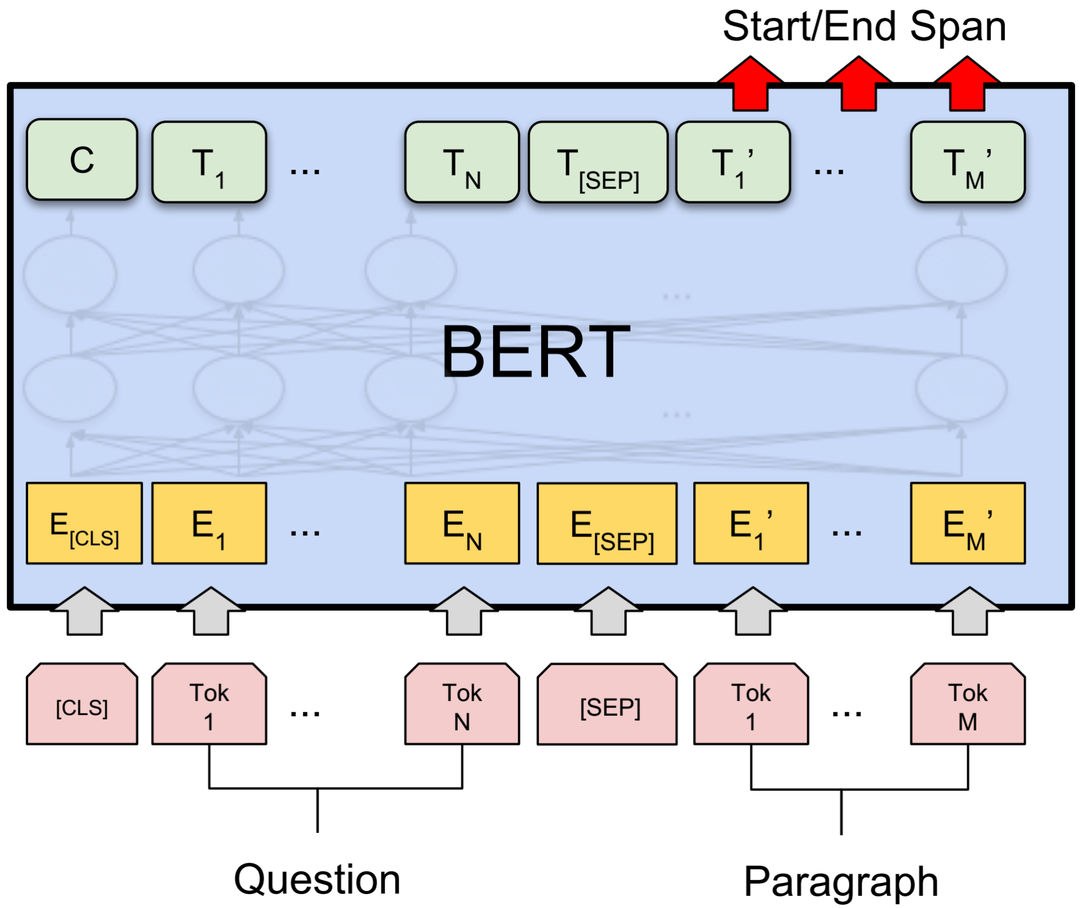


(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

- Add linear classifier on top of token hidden representations.

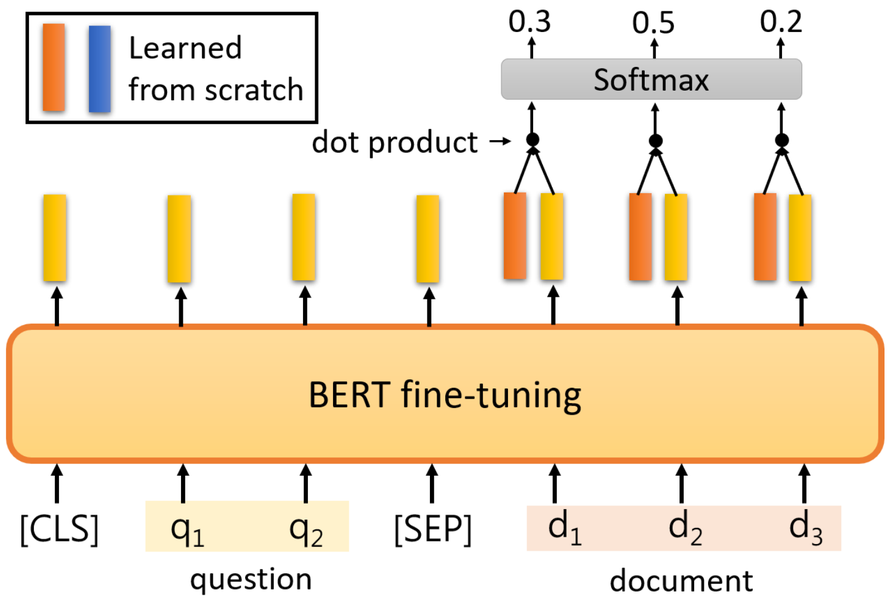
Applications: Token-Level Tasks

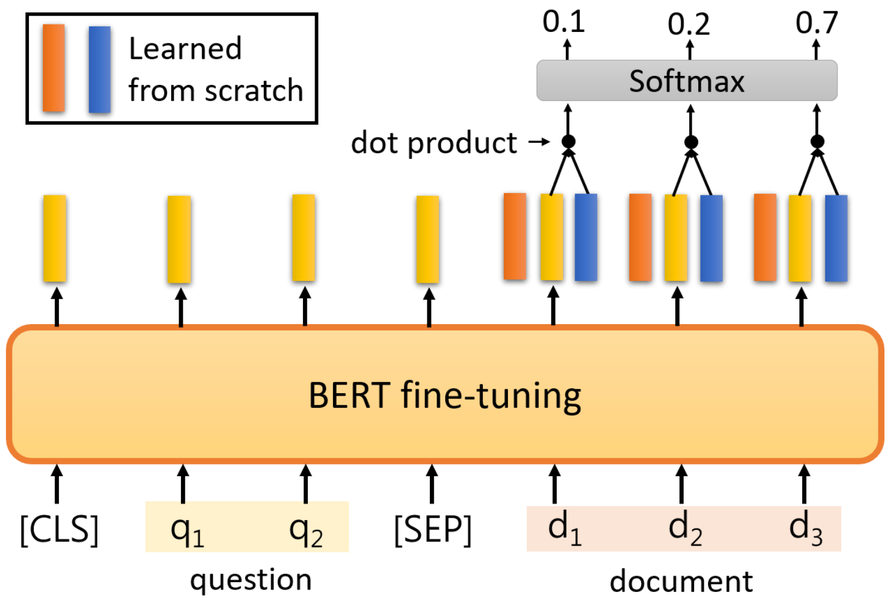
- **Application**: extraction-based question answering.
- **Input**: a question and a paragraph.
- **Output**: start and end positions of the answer in the paragraph.



Applications: Token-Level Tasks

- **Loss function**: cross-entropy for start and end positions.
- Learn **two vectors** (orange and blue) from scratch, one for helping determine *start* position, the other for *end* position.
- e.g., the question's answer is "d2 d3" (s = 2, e = 3).





BERT: Downstream Performance

- BERT (after fine-tuning) achieves SOTA on tasks including:
  - CoLA (Corpus of Linguistic Acceptability)
  - STS-B (Semantic Textual Similarity)
  - RTE (Recognizing Textual Entailment)
  - MRPC (Microsoft Research Paraphrase Corpus)
  - QQP (Paraphrase Detection)
  - QNLI (QA-based natural language inference)
  - SST-2 (Sentiment Analysis)

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

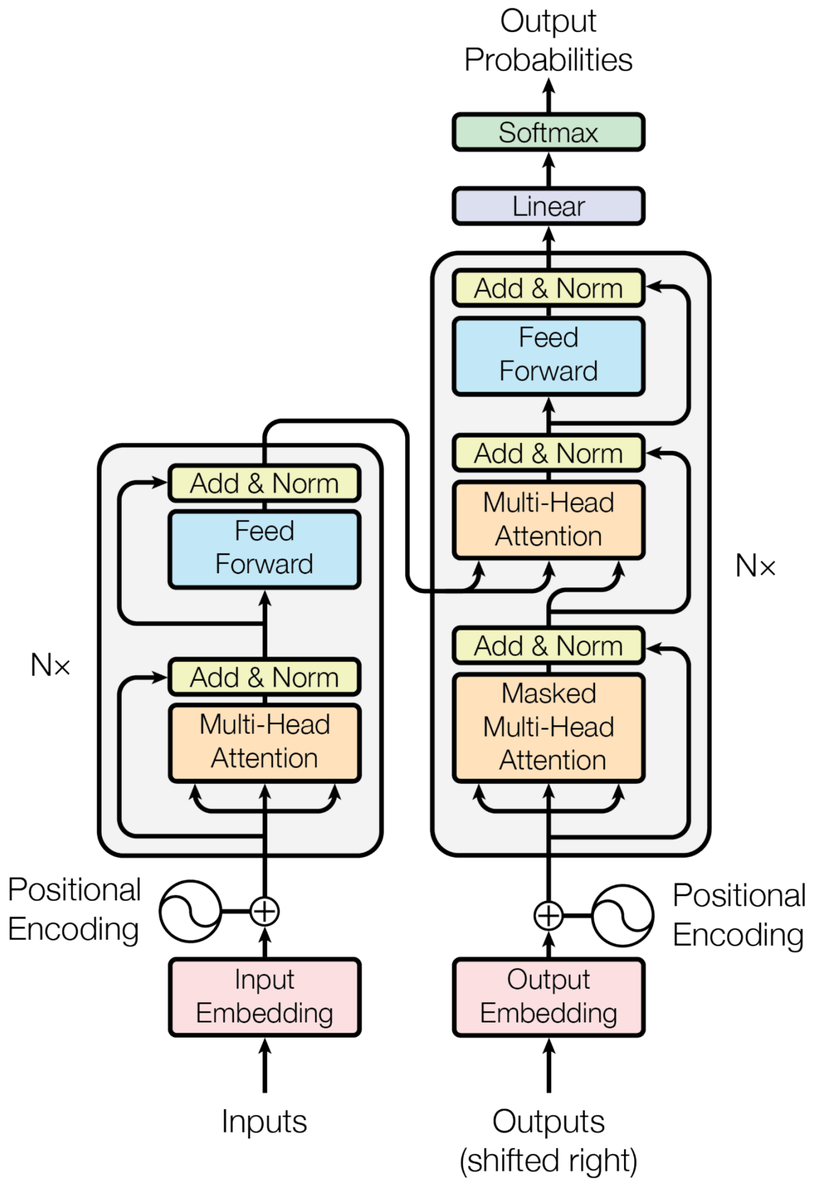
Outline

- ❑ Pretraining
- ❑ Groundbreaking: BERT
- ❑ **Encoder-Decoder Models**
- ❑ Decoder-Only: GPTs
- ❑ Practice & Discussion

Encoder-Decoder Models: T5/BART

T5/BART

- Definition: Pretrained generative models based on the Transformer Encoder-Decoder, unifies NLP problems into a **text-to-text** learning format.
- Training: Uses large-scale unsupervised corpus through various self-supervised tasks (e.g., text filling, text reconstruction, sentence ordering) to pretrain, then adapts to *downstream* tasks.

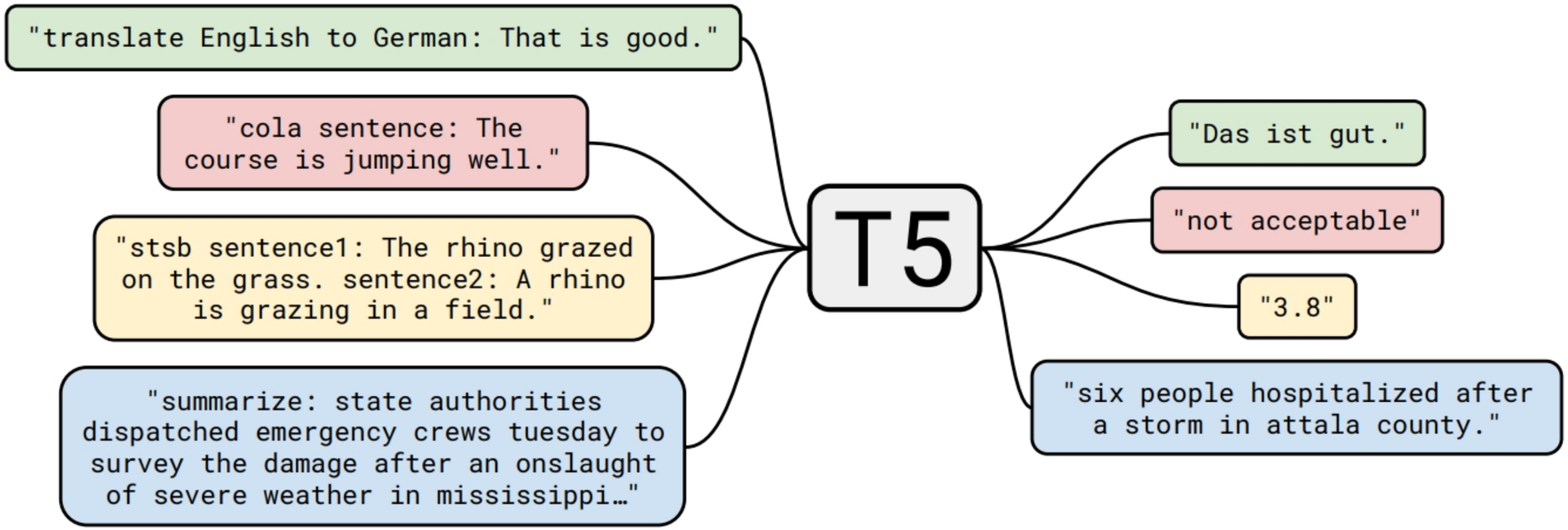


Encoder-Decoder Models: T5/BART

- Key Features:
  - Contains both Encoder and Decoder: Encoder encodes input context; Decoder autoregressively generates output.
  - Supports multiple self-supervised pretraining objectives: e.g., Span Corruption (T5), Text Infilling (BART).
  - Highly flexible downstream tasks: Solve classification, generation, and other tasks using the unified "input text → output text" paradigm.

Encoder-Decoder Models: T5

- **T5** (**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer)
  - One format for all tasks!
    - Question answering, language modeling, span extraction, etc.
  - [Task-specific prefix]: [Input text] → [Output text]

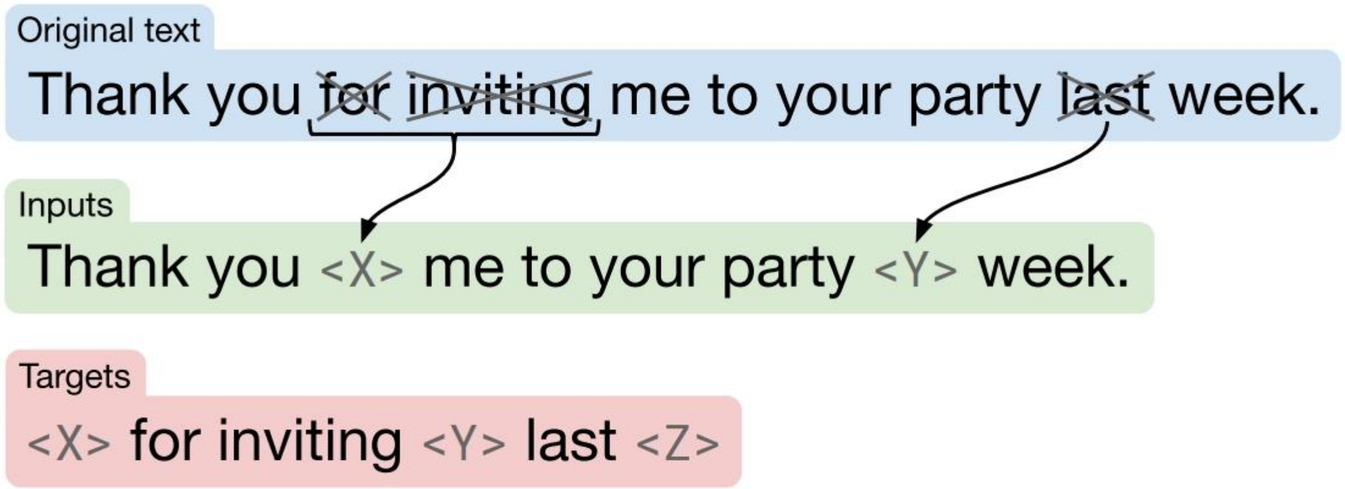


Encoder-Decoder Models: T5

- **T5** (**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer)
  - Parameter scale: Multiple versions available (Base, Large, 3B, 11B), ranging from 220 million to 11 billion parameters.
  - Training data: Uses C4 (Colossal Clean Crawled Corpus)
    - High-quality web pages filtered from Common Crawl, with noise removed, retaining approximately 750GB of plain text.
    - Covers diverse topics, supporting multitask pretraining.

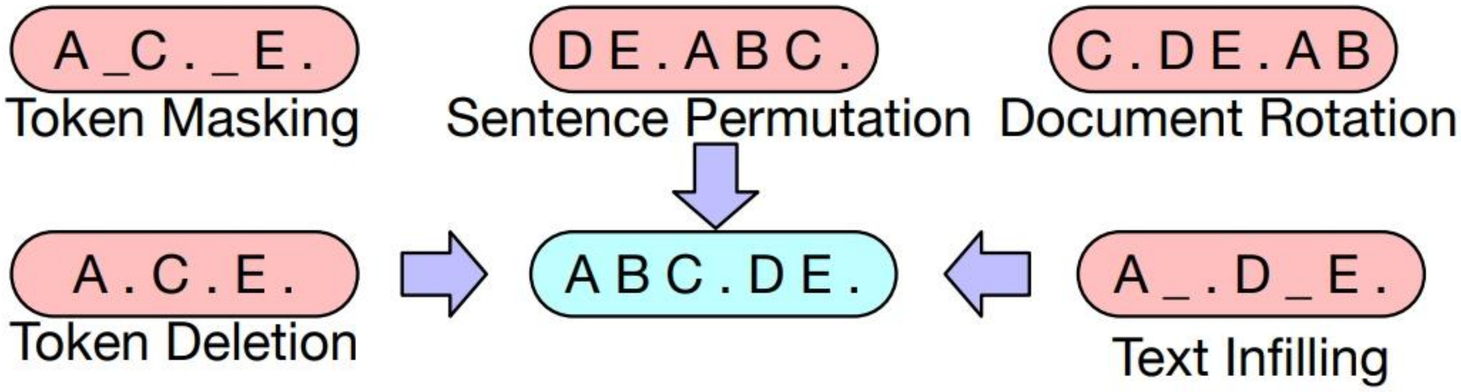
Encoder-Decoder Models: T5

- **Pretraining objective**
  - Span Corruption: Randomly mask contiguous spans in the input; the model must reconstruct the full span in the output.



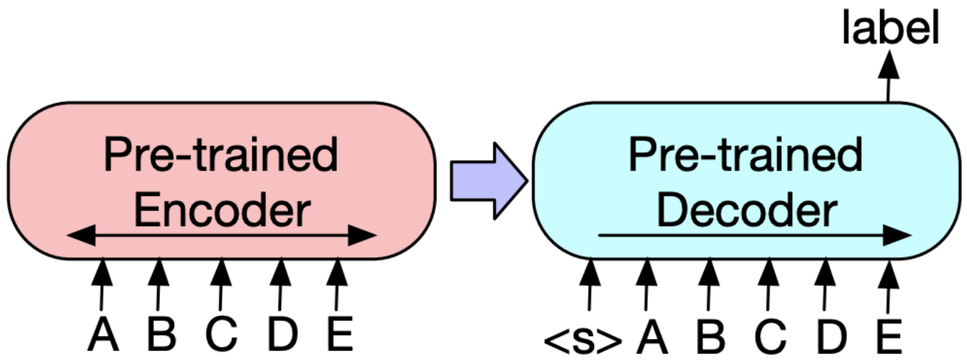
Encoder-Decoder Models: BART

- **BART** (Denoising Sequence-to-Sequence Pre-training)
  - Transformer-based encoder-decoder structure combining discriminative and generative training.
  - Training criterion:
    - Apply an arbitrary noise function to perturb (corrupt) the text.
    - Learn a model to reconstruct the original text.

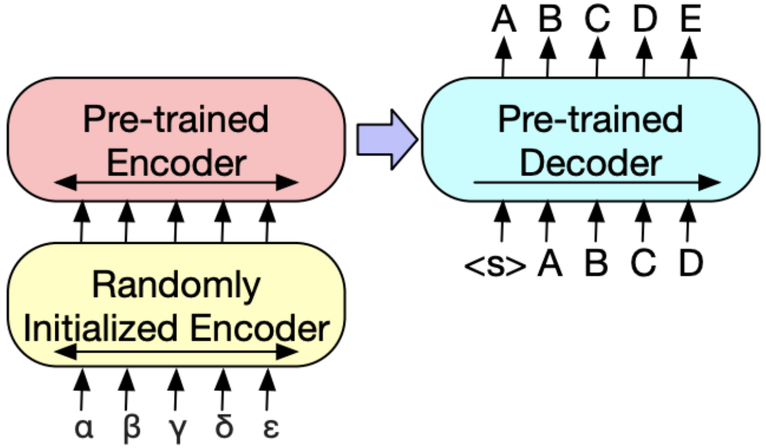


Encoder-Decoder Models: BART

- **Fine-tuning**:
  - Classification tasks: The same input is fed simultaneously into BART's encoder and decoder.
  - Machine translation: Introduce an additional small encoder to replace BART's word embeddings, then train it to map foreign language words into inputs that BART can denoise into English.



Classification Tasks



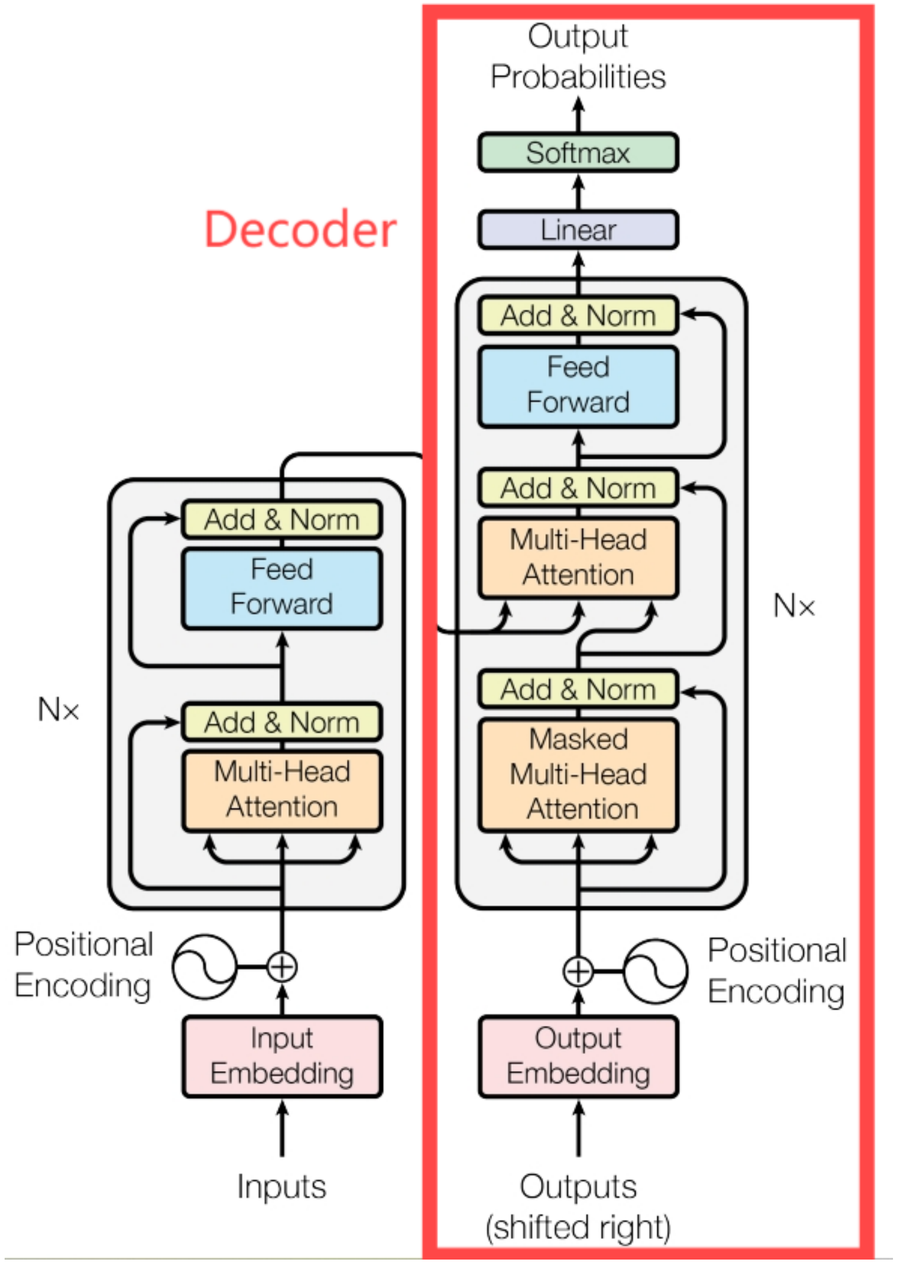
Machine Translation

Outline

- ❑ Pretraining
- ❑ Groundbreaking: BERT
- ❑ Encoder-Decoder Models
- ❑ **Decoder-Only: GPTs**
- ❑ Practice & Discussion

Decoder-Only Models: GPT

- **GPT:** Generative Pretrained Transformer
  - Definition: Transformer decoder-based pretrained model, learns unidirectional semantic information from context.
  - Training: Uses large-scale unsupervised corpus for pretraining (captures context), then adapts to downstream NLP tasks.



Decoder-Only Models: GPT

- **Core**: Deep Transformer decoder + unidirectional context learning.

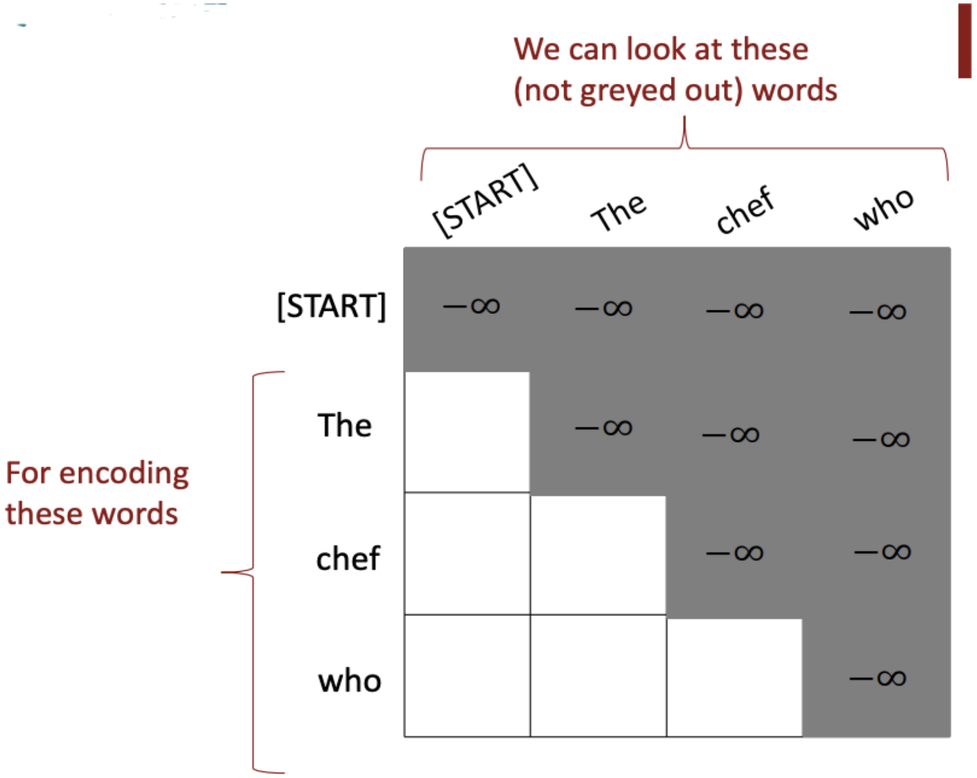
> **Question**: Why unidirectional?
>
> **Answer**: Generative tasks require relying only on left context.

- **Example**:
  - "We went to the store."
  - "I want to eat pizza."

Decoder-Only Models: GPT

Masked Multi-Head Attention

- To use self-attention in decoders, we need to ensure can't peek at the future.
- At every timestep, we could change the set of keys and queries to include only past words. (Inefficient!)
- To enable parallelization, we mask out attention to future words by setting attention scores to $-\infty$.



$$e_{ij} = \begin{cases} q_i^T k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$

Decoder-Only Models: GPT

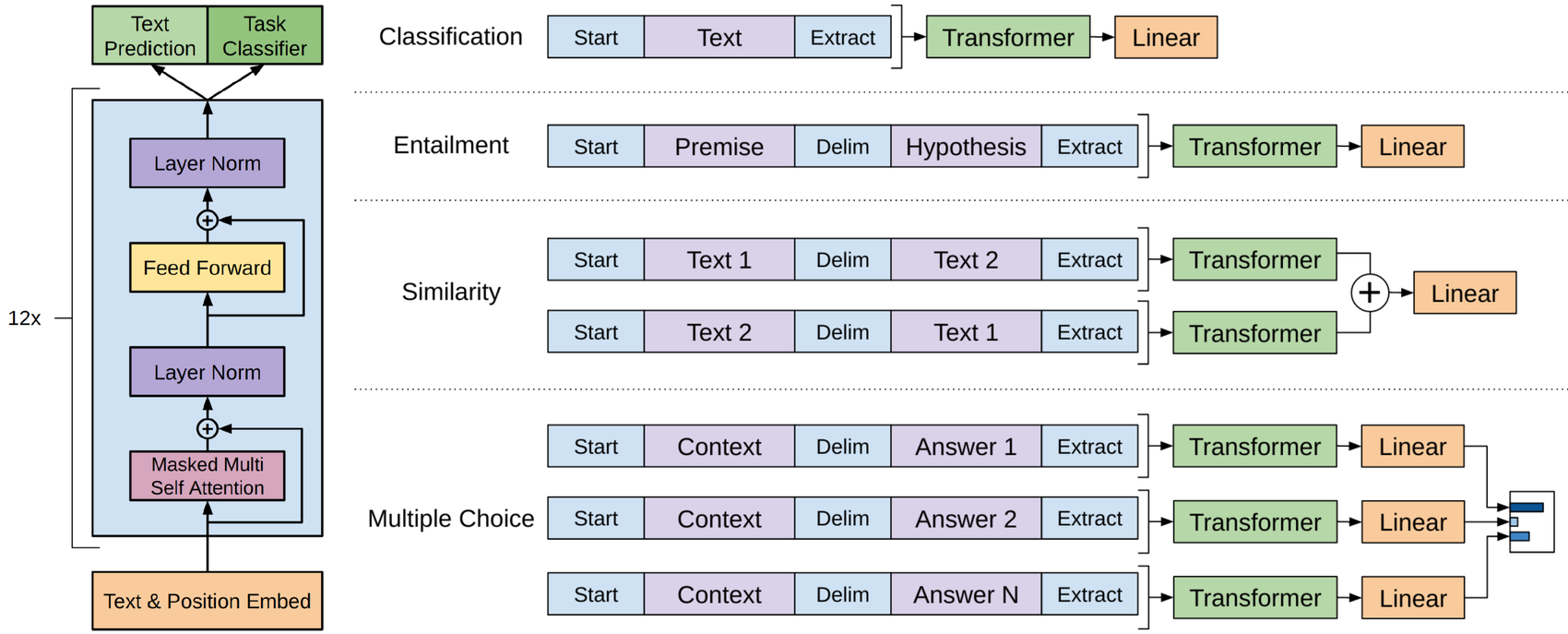
- **Details GPT:**
  - Uses 12 layers of Transformer decoder, 117 million parameters.
  - Hidden state dimension: 768; Feed-forward hidden layer dimension: 3072.
  - Uses subword units (BPE) for tokenization.
  - Training corpus: *BooksCorpus*, containing over 7,000 unique books.
    - BooksCorpus contains long, continuous text, aiding the learning of long-range dependencies.

Decoder-Only Models: GPT

- **Pretraining Task**: Causal Language Modeling
  - **Goal**: Build a generative model to predict the next word given a context.
    - Input: A sequence of text.
    - Target: Predict the next word.
    - Training: The model generates a probability distribution based on the preceding words, then optimizes the model parameters to make the *prediction closer to the actual word*.
  - **Advantage**: The labels (i.e., the next word) naturally exist within the data, allowing training to leverage massive amounts of unlabeled text.

Decoder-Only Models: GPT

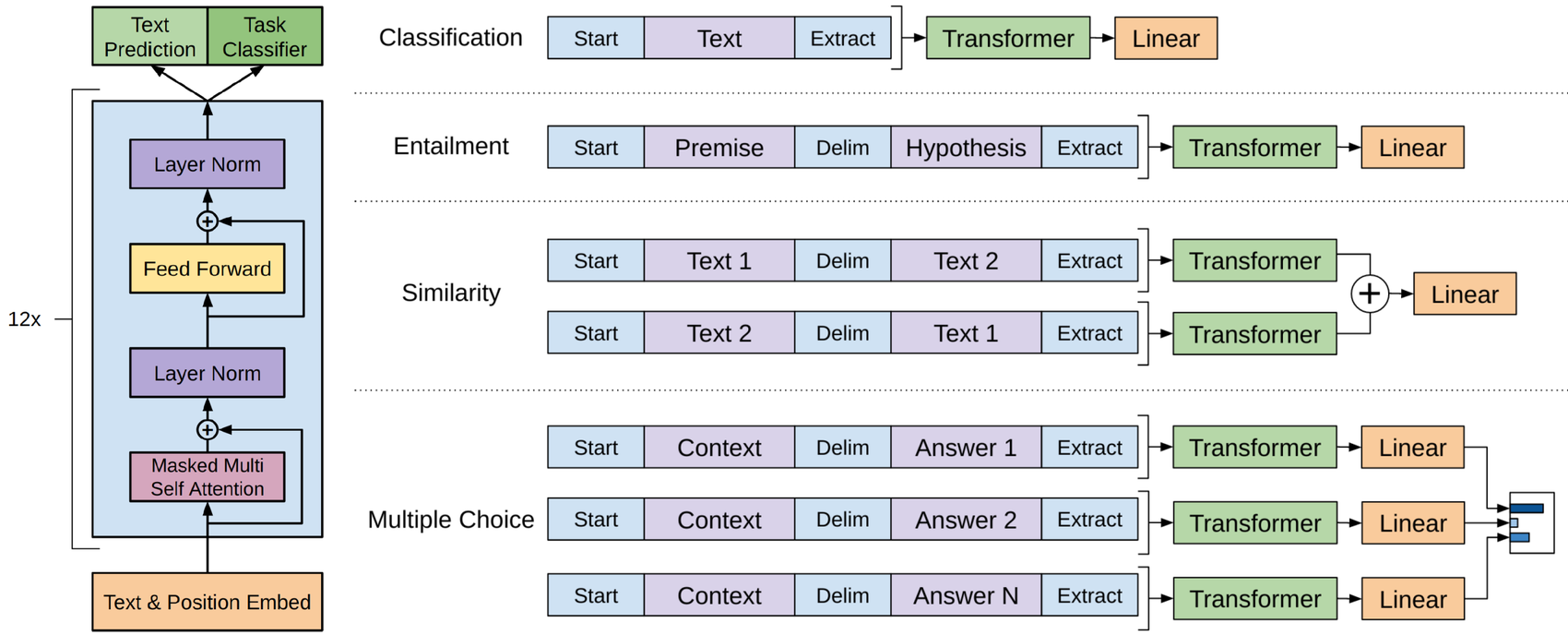
- **Fine-tuning approach:**
  - Downstream tasks are converted into "*text-to-text prediction problems*".
  - *Inputs* are concatenated and uniformly encoded by the Transformer.
  - A simple *linear classifier* is added for task-specific outputs.



Decoder-Only Models: GPT

❑ Classification Tasks

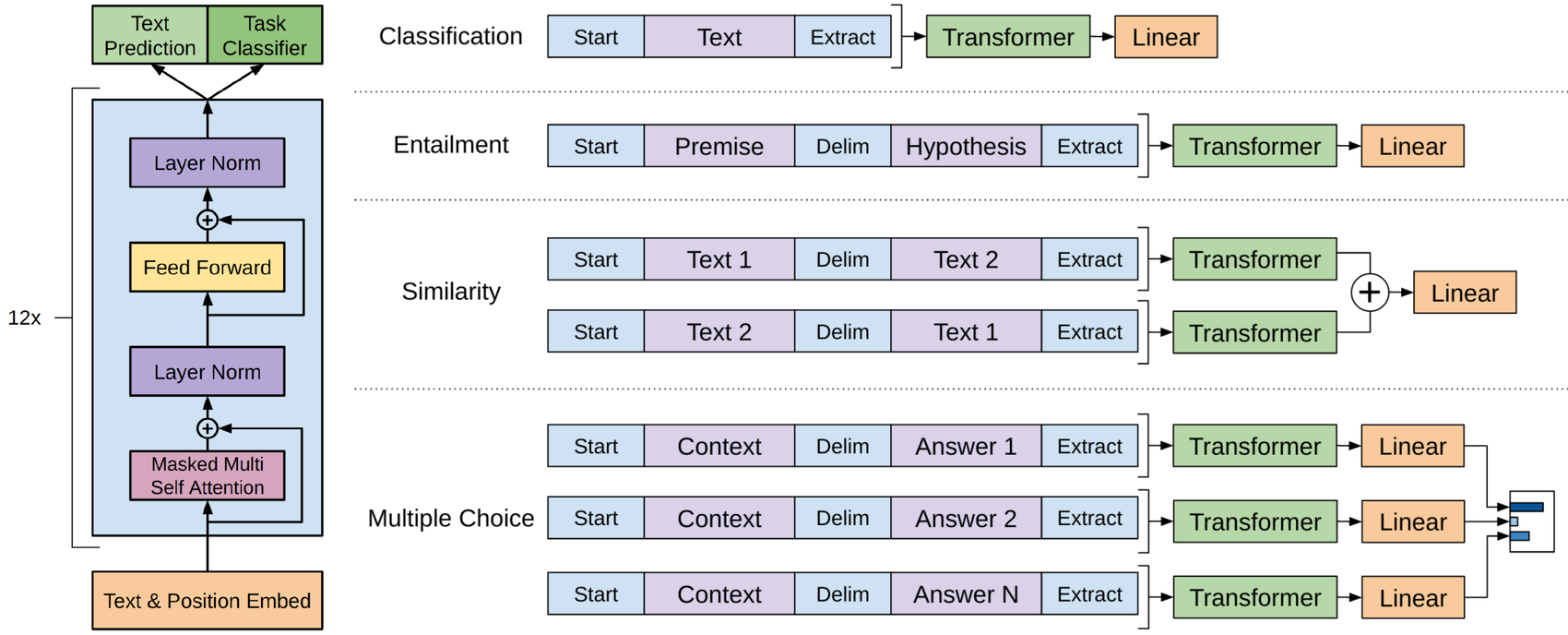
- Input format: [Start] + Text + [Extract]
- Output: Use Transformer to extract sentence vectors, then use a Linear layer to output classification results (e.g., sentiment analysis).



Decoder-Only Models: GPT

❑ Entailment/Natural Language Inference

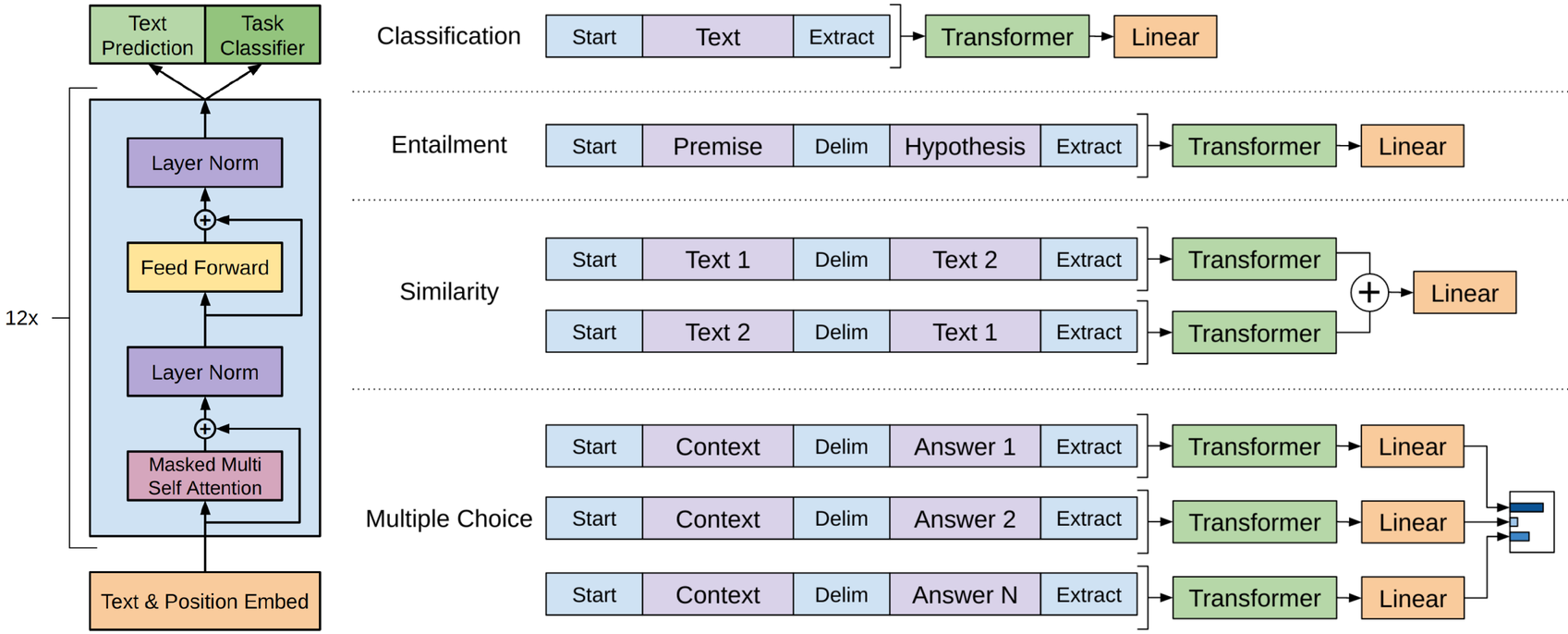
- Input format: [Start] + Premise + [Delim] + Hypothesis + [Extract]
- Two sentences are concatenated, separated by a special delimiter [Delim]. The model determines their relationship (*entailment/contradiction/neutral*).



Decoder-Only Models: GPT

Similarity

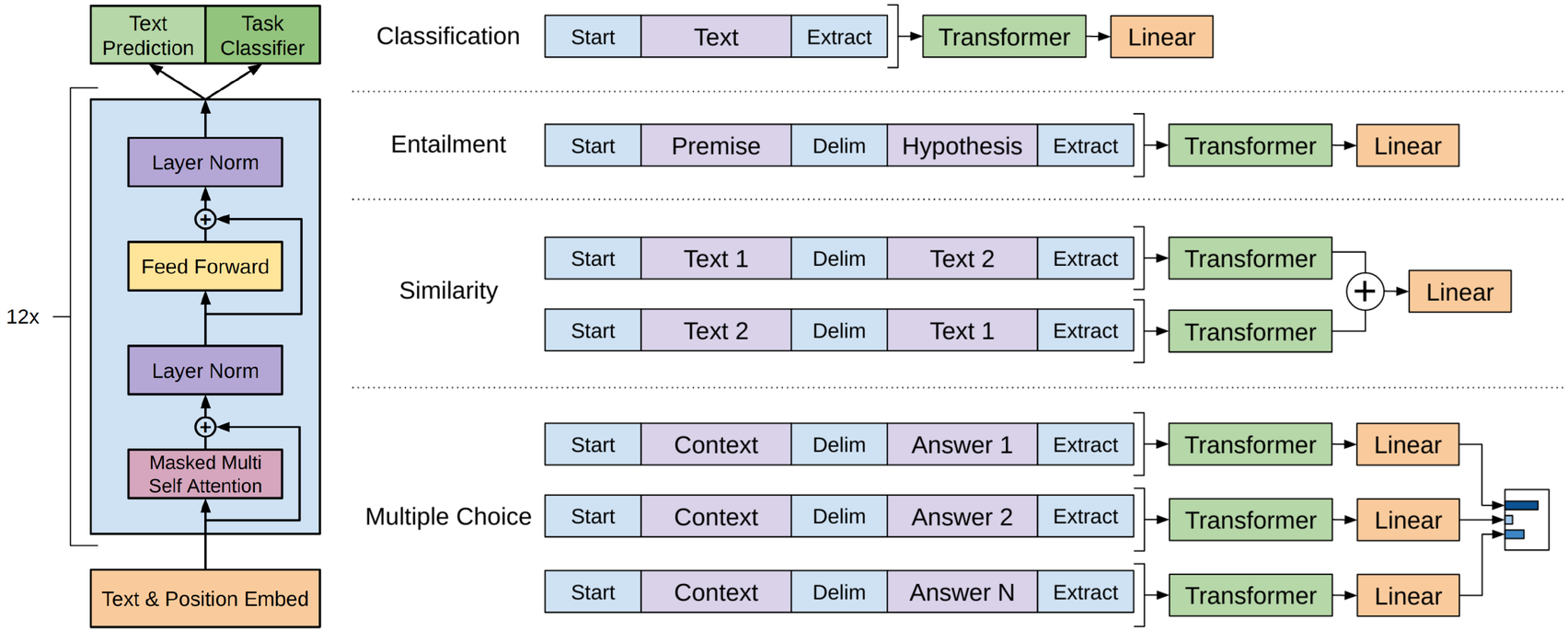
- Input: Two sentences are fed into the Transformer separately. The generated vectors are *concatenated* or *compared* for similarity, followed by a *Linear* layer to predict a similarity score.



Decoder-Only Models: GPT

❑ Multiple Choice

- Input format: [Start] + Context + [Delim] + Answer *i* + [Extract]
- Construct one input for each candidate answer. Pass each through GPT to output a score. **Select the option with the highest score**.



Decoder-Only Models: GPT

❑ Key Characteristics

- **Unified Architecture**: All tasks reuse the same pretrained decoder structure, requiring only a lightweight linear layer on top.
- **Flexible Input Format**: Can concatenate context, questions, options, etc., allowing the model to "understand" and generate the correct output.
- **Efficient Adaptation**: Can adapt to various NLP downstream tasks with minimal fine-tuning.

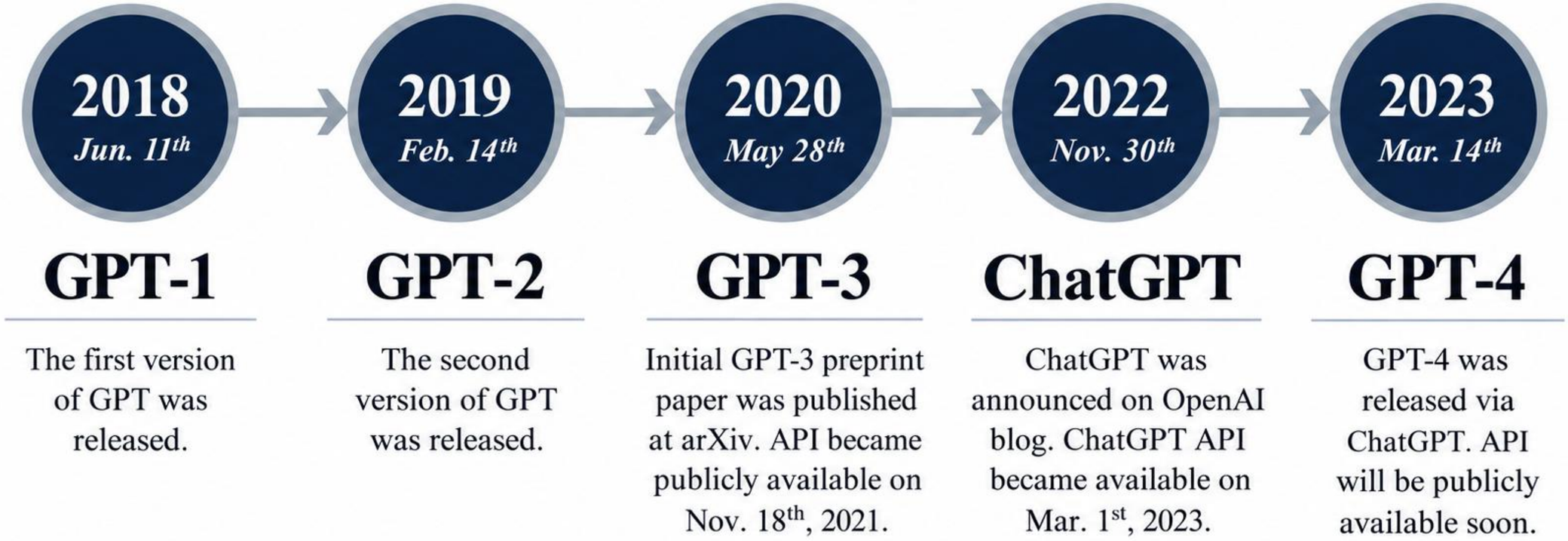
Decoder-Only Models: GPT

- **Strengths**:
  - Exceptional at natural language generation tasks.
  - Can produce fluent, coherent, and creative text. Massive scale enables impressive **zero-shot and few-shot** learning via prompting.
  - Highly versatile (多才多艺).

Decoder-Only Models: GPT

- **Weaknesses**:
  - While they develop strong understanding through scale, their unidirectional nature during generation might be less optimal for tasks requiring deep bidirectional analysis compared to encoders.
  - Can be prone to hallucinations or lose coherence over very long outputs.
  - Inference can be slower due to the sequential generation process.

GPT Series Development



GPT Series Development: GPT-1

- **GPT-1** (first version)
  - Backbone: 12 blocks, 12 masked self-attention heads, 64-dimensional output for each input token.
  - Training set: Part of BooksCorpus (over 7000 fiction books).
  - Training objective: Pretrain by predicting next word conditioned on previous words, then fine-tune for discriminative tasks.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

GPT Series Development: GPT-2

- **GPT-2**: larger dataset and model
  - Training set and model parameters increased by a factor of 10.
  - 1.5 billion parameters, dataset of 8 million docs (40GB).
  - Better performance on downstream generative tasks with smaller dataset: machine translation, QA, summarizing, etc.

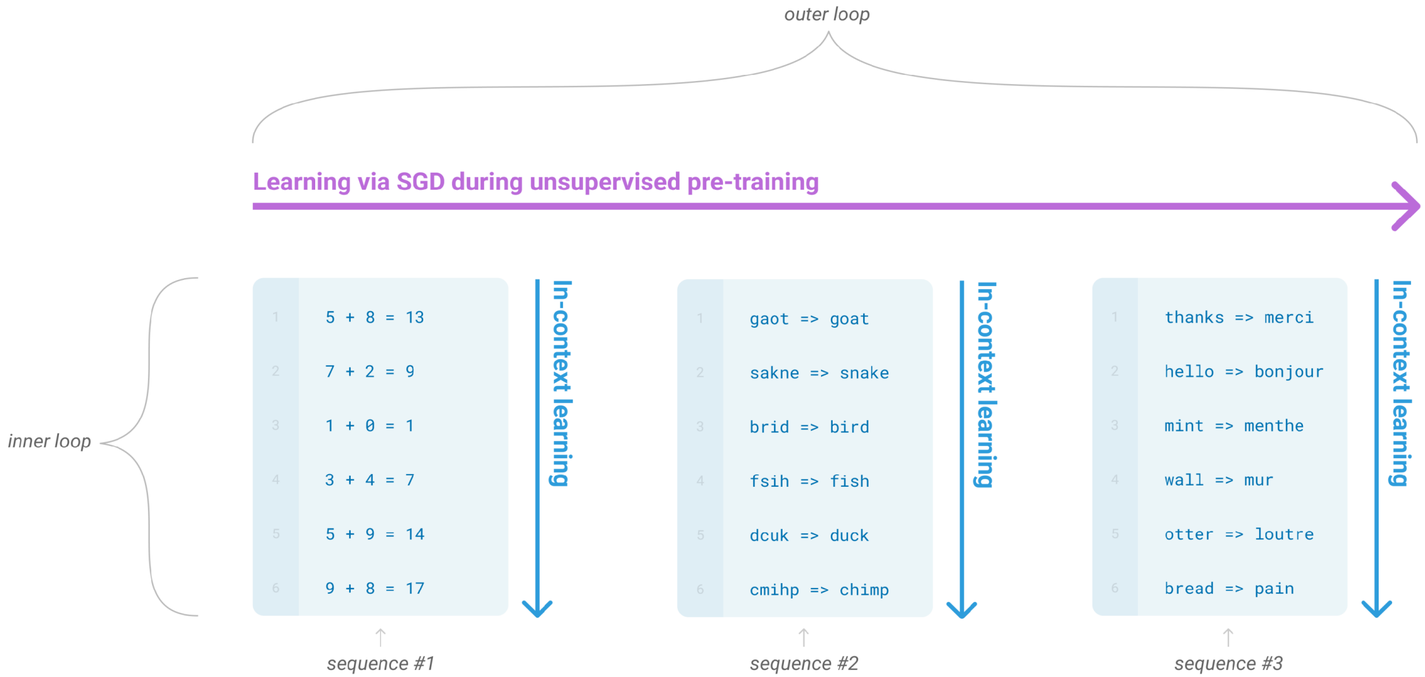
GPT Series Development: GPT-3

- **GPT-3**: larger dataset and model, multi-task training
  - **175 billion** parameters! ~570GB of text.
  - 96 blocks; 96 heads; 12888-dim for each token.
  - Cost $12 million to train, 355 years of V100 GPU time.
  - Long context window (2048 tokens) in input sequence.
  - Downstream tasks: no fine-tuning (so no gradient update).
  - Perform many different NLP tasks through flexible prompting.

GPT Series Development: GPT-3

❑ Multi-task training by GPT-3

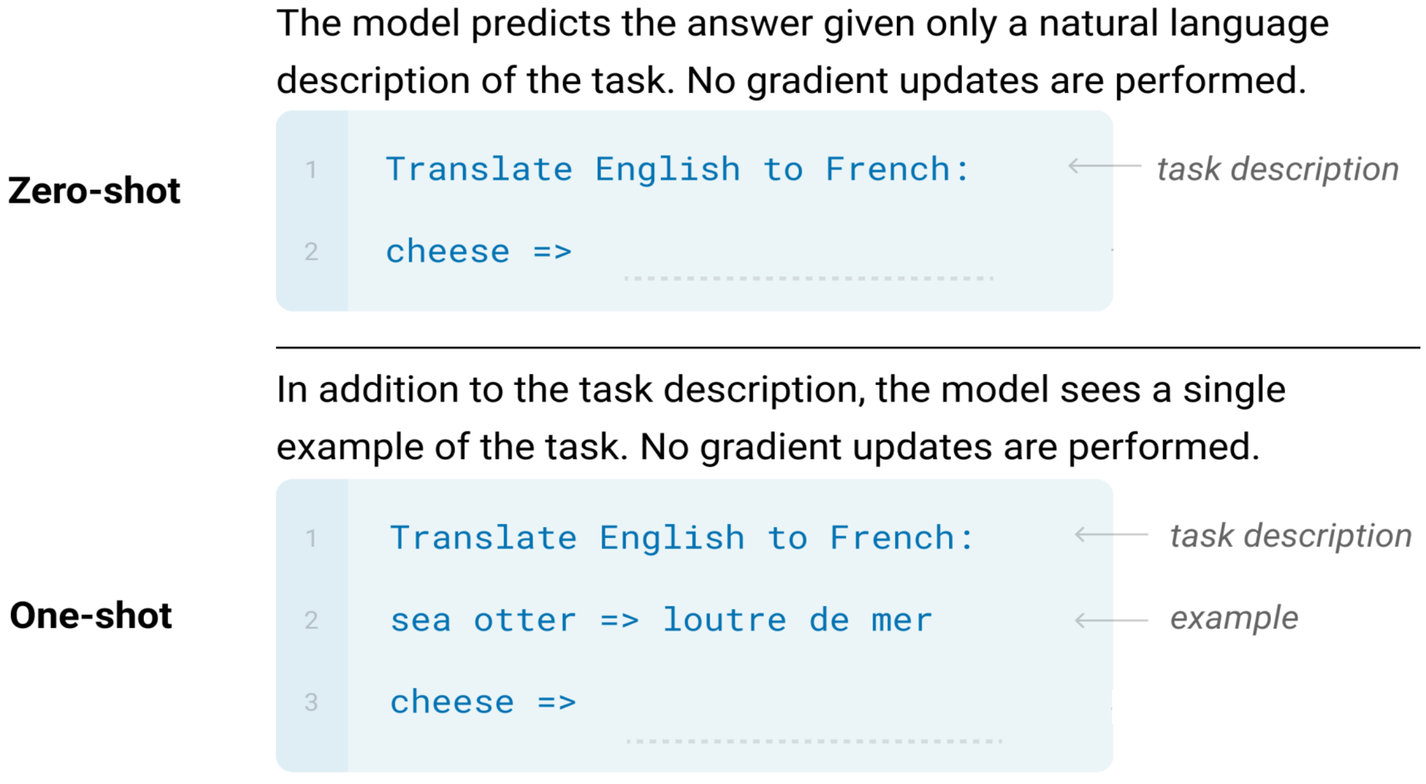
- **Translation**: *(translate English to French, English text, French text).*
- **Reading compreh.**: *(answer the question, document, question, answer).*
- Can be considered as **prompt**-aided model training! Task description as the prompt.



GPT Series Development: GPT-2/GPT-3

❑ Usage of GPT3

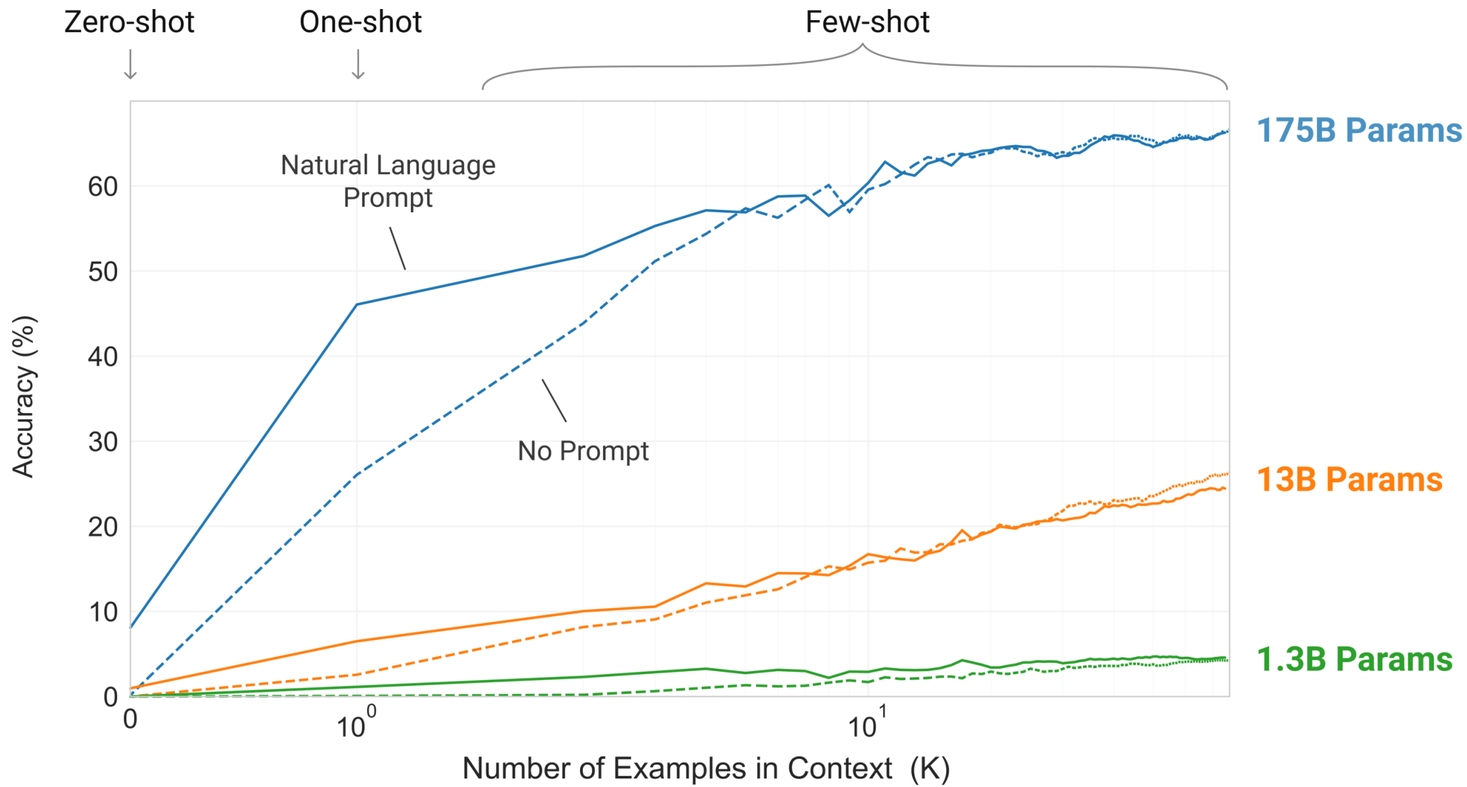
- Can perform *zero-shot*, *one-shot*, *few-shot learning*.
- No fine-tuning on downstream tasks.
- Prompt in input sequence tells model which task to perform.



GPT Series Development: GPT-3

❑ Performance of GPT3

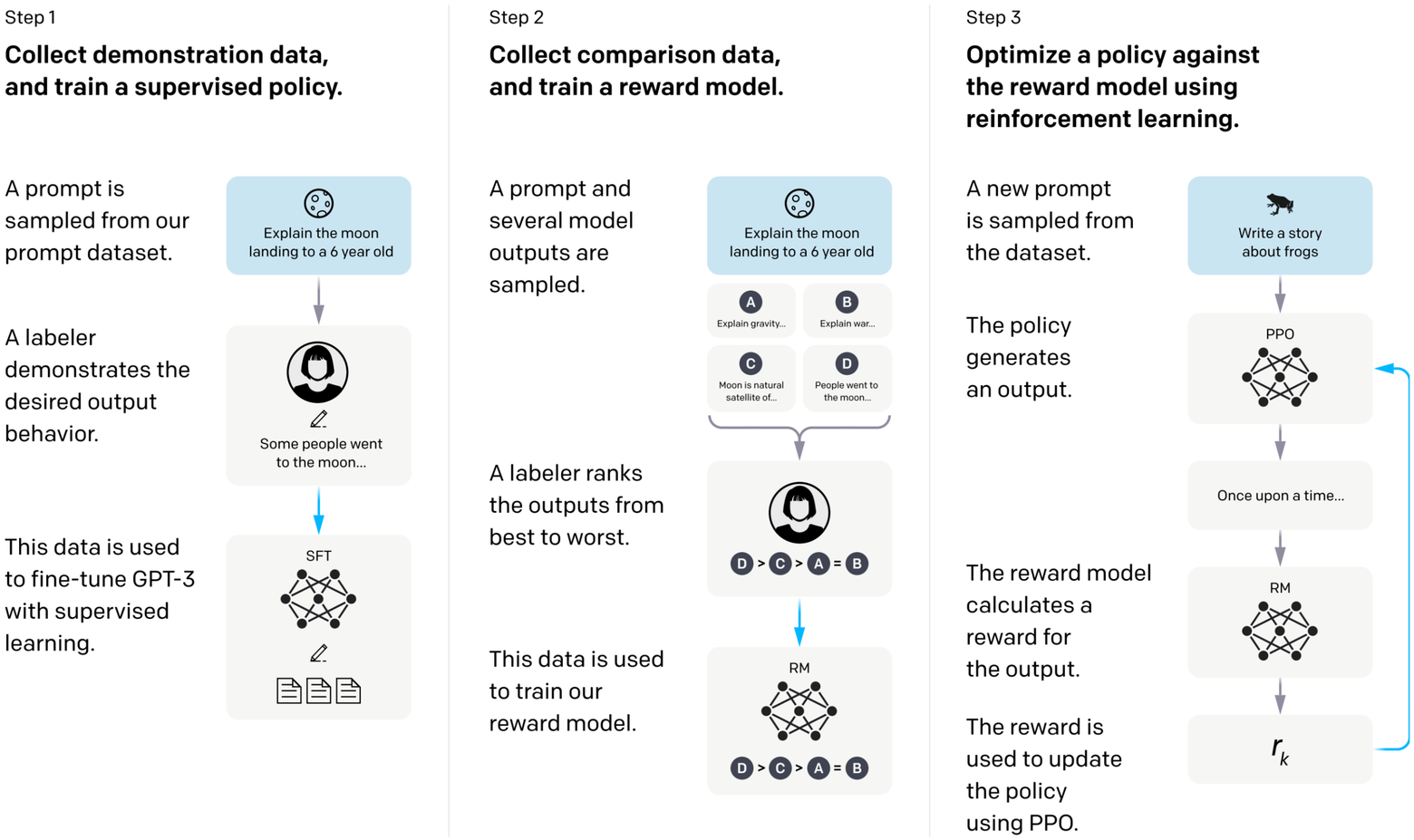
- Larger model works much better
- *Prompt* (task description) is crucial particularly for one-shot
- Why is *prompt* not necessary for few-shot task?



GPT Series Development: ChatGPT

❑ ChatGPT: built on InstructGPT (GPT3.5)

- Reinforcement learning from human feedback (RLHF)



GPT Series Development: ChatGPT

- **ChatGPT** vs. **earlier**: less harmful and untruthful output.
- What ChatGPT can do
  - Chatbot: chat with others.
  - Write and debug computer programs.
  - Compose music, teleplays, fairy tales, and student essays.
  - Answer test questions.
  - Write poetry and song lyrics.
  - Emulate a Linux system, simulate an entire chat room.
  - Play games.

GPT Series Development: ChatGPT

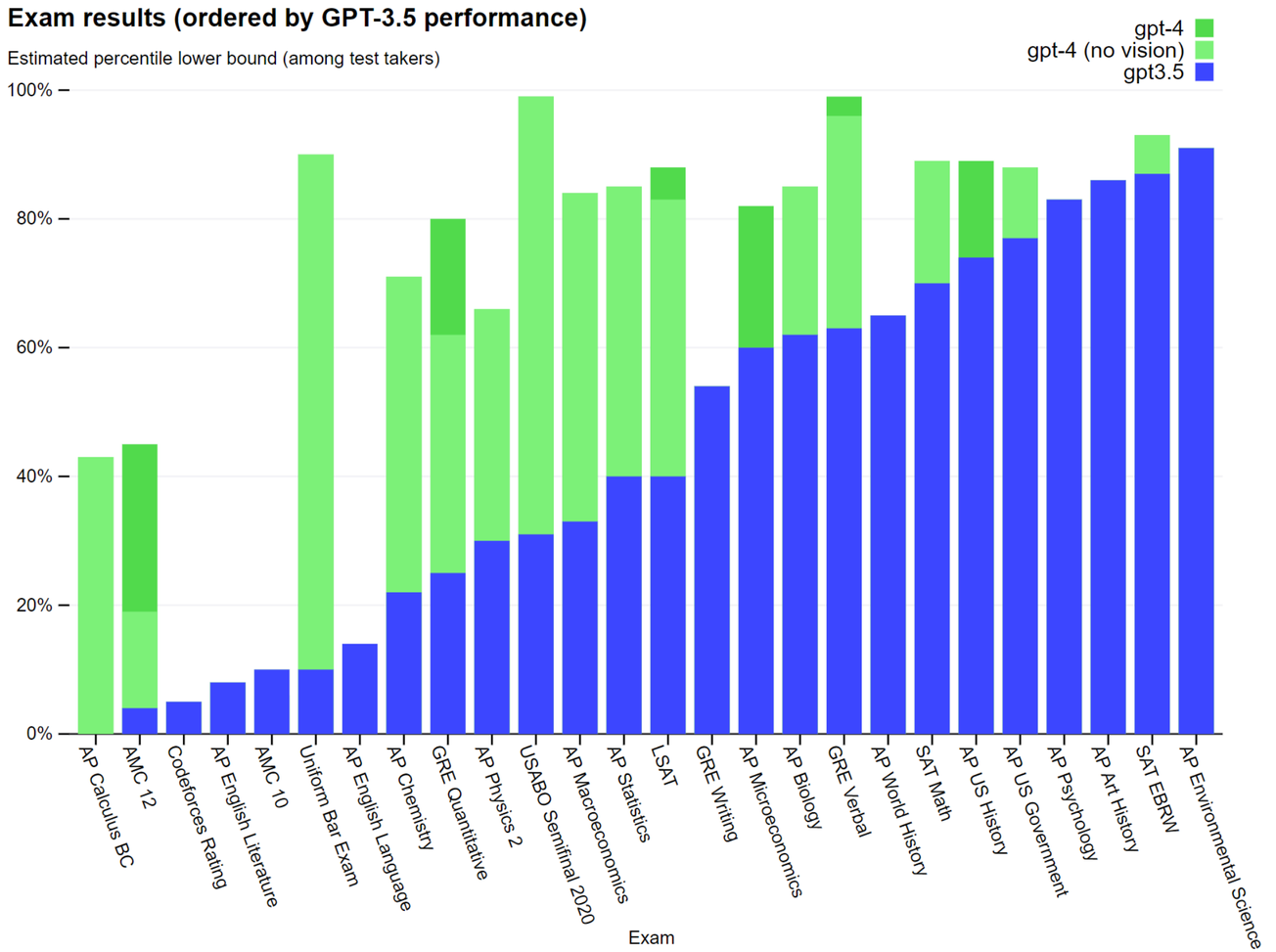
- **ChatGPT limitations**
  - Sometimes writes plausible-sounding but incorrect answers.
  - Sensitive to tweaks to the input phrasing.
  - Often excessively verbose and overuses certain phrases.
  - May respond to harmful query or exhibit biased behavior.

GPT Series Development: GPT-4

- **GPT-4**: released in March 2023
  - Multimodal large language model.
  - Input can be images and text; output is text.
  - Technique details not revealed, even not know size.
  - More reliable, creative, and smarter than GPT-3.5.
  - Two versions: with context windows of 8,192 and 32,768 tokens.
  - "System message": specify response style/tone.
  - Alignment with human values and safety: refuse to answer harmful prompts.

GPT Series Development: GPT-4

- GPT-4 outperforms GPT3.5 (ChatGPT) significantly.
  - Outperform most students in many tests.



GPT Series Development: GPT-5

GPT-5: Next-Generation AI Language Model

- **Reasoning**: Improved multi-step problem solving and logical consistency.
- **Accuracy**: More reliable factual responses and reduced hallucinations.
- **Multimodality**: Better integration of text, images, audio, and tools.
- **Customization**: Fine-tuned behaviors for specific industries and workflows.
- **Safety & Alignment**: Stronger safeguards and transparent reasoning features.
- **Adaptability**: Faster responses and improved handling of complex tasks.

Outline

- ❑ Pretraining
- ❑ Groundbreaking: BERT
- ❑ Encoder-Decoder Models
- ❑ Decoder-Only: GPTs
- ❑ **Practice & Discussion**

Practice & Discussion: The Architecture Debate

Which Language Model Architecture Is the Best?

Three teams. Three architectures. One debate.

Team A	Team B	Team C
Encoder-Only	Encoder–Decoder	Decoder-Only
BERT	T5	GPT

- Your Mission
  - Defend your assigned architecture and convince the class that it is the best choice for building language models.
- Consider:
  - Capabilities: What tasks can it perform well?
  - Efficiency: What are its training and inference costs?
  - Scalability: How well does it scale to large models?
  - Limitations: What are its main weaknesses?
  - Future Potential: Why should we continue investing in it?


Learn something useful.
Enjoy the process.